

Article

CNNRes-DIndRNN: A New Method for Detecting TLS-Encrypted Malicious Traffic

Jinsha Zhang ^{1,2} , Xiaoying Wang ^{1,2,*}, Chunhui Li ^{1,2}, Qingjie Zhang ^{1,2}, Guoqing Yang ^{1,2}, Xinyu Li ^{1,2} , Fangfang Cui ^{1,2} , Ruize Gu ^{1,2} , Panpan Qi ^{1,2} and Shuai Liu ^{1,2}

¹ School of Computer Science and Engineering, Institute of Disaster Prevention, Langfang 065201, China; 23661602@st.cidp.edu.cn (J.Z.); chunhuili2023@163.com (C.L.); zhangqingjie32@163.com (Q.Z.); 23661622@st.cidp.edu.cn (G.Y.); 23661623@st.cidp.edu.cn (X.L.); 24661604@st.cidp.edu.cn (F.C.); 24661605@st.cidp.edu.cn (R.G.); 25661613@st.cidp.edu.cn (P.Q.); facai9268@gmail.com (S.L.)

² Langfang Key Laboratory of Network Emergency Protection and Network Security, Langfang 065201, China

* Correspondence: wangxiaoying@cidp.edu.cn

Abstract

While ensuring the accuracy of encrypted malicious traffic detection, improving model training speed remains a challenge. In order to solve this challenge, we propose CNNRes-DIndRNN for detecting encrypted malicious traffic classification. This model uses 1D-CNN to capture local feature relationships between data and IndRNN to capture their global dependency relationships. This method uses Zeek (version 7.0.0) to filter TLS datasets and NetTiSA to build time-series features that help models identify malicious behaviors. Combine time-series and encrypted features, then encode them with XLNet to improve model learning ability and speed training. In the final step, the encoded data is fed into CNNRes-DIndRNN. The results on five datasets including CTU-13 and MCFP showed that CNNRes-DIndRNN achieved 99.81% accuracy in binary classification and 99.67% in multi-class classification. These results represent improvements of 0.50–7.78% (binary) and 0.93–12.26% (multi-class) over all baseline methods. In performance comparisons, CNNRes-DIndRNN achieved the fastest training and testing times. It achieves the best comprehensive performance while maintaining high recognition accuracy.

Keywords: encrypted malicious traffic classification; convolutional neural networks; recurrent neural networks; time-series algorithm; SSL/TLS

1. Introduction

Encrypted malicious traffic refers to network traffic generated by attackers. These attackers exploit cryptographic protocols to carry out malicious activities. Although designed to ensure data confidentiality and transmission integrity, these protocols are repurposed by attackers to conceal cyber attacks and enhance stealth. By leveraging these protocols, attackers can exploit encrypted channels to conduct malicious command and control (C&C), including data exfiltration, bypassing firewalls or intrusion detection systems (IDS), and masquerading malicious traffic as legitimate. These activities pose critical threats to internet security.

Check Point's 2025 Report [1] documents a 44% global surge in cyberattack, with ransomware shifting from encryption to direct data theft and extortion—impacting healthcare severely, which saw a 47% increase and ranked as the second most targeted sector. Meanwhile, over 200,000 edge devices were hijacked by botnets, and 96% of 2024 exploits targeted vulnerabilities disclosed the previous year, revealing critical patch management flaws.



Academic Editor: Carlo Blundo

Received: 19 November 2025

Revised: 12 December 2025

Accepted: 17 December 2025

Published: 24 December 2025

Copyright: © 2025 by the authors.

Licensee MDPI, Basel, Switzerland.

This article is an open access article distributed under the terms and

conditions of the [Creative Commons](https://creativecommons.org/licenses/by/4.0/)

[Attribution \(CC BY\)](https://creativecommons.org/licenses/by/4.0/) license.

Zscaler's 2023 Crypto Attacks Report [2] documents a 24% year-over-year rise in HTTPS-based threats on its cloud platform. Crypto-malware and malicious content accounted for 78% of all observed attacks. The manufacturing sector, processing 2.1 billion AI/ML transactions, was targeted in 32% of encryption-based incidents. Browser exploits and ad spyware surged by 297% and 290%. 86% of 30 billion threats detected were transmitted via encrypted channels, including malware, ransomware, and phishing. This study explores encrypted malicious traffic classification and confirms the proposed method's effectiveness through validation.

Current research on encrypted malicious network traffic classification can be categorized into two approaches. The first methodological approach achieves superior detection accuracy, typically between 97% and 99.90%. However, it requires significant processing time during training and testing, making it unsuitable for real-time deployment—especially in resource-limited environments. The second methodological approach prioritizes efficiency during training and testing. However, it achieves significantly lower accuracy—typically around 90% or less—which may be insufficient for demanding monitoring scenarios.

To resolve the aforementioned issues, this paper proposes a deep learning-based encrypted malicious traffic classification method named CNNRes-DIndRNN (Residual Convolutional Neural Network—Dual-layer Independent Recurrent Neural Network). This method improves the speed of model training and testing while ensuring detection and classification accuracy.

The contributions of this paper are threefold:

- (1) We have developed ENNetTSd-2025 (Encrypted Network Traffic Dataset-2025) by aggregating five public datasets: CIC-IDS2017 [3], MCFP [4], CTU-13 [5], VPN-NonVPN [6], and USTC-TF2016 [7]. TLS-encrypted traffic has been filtered using Zeek [8] to generate a dataset focused exclusively on encrypted flows.
- (2) We have used a time-series algorithm (NetTiSA [9]) to extract time-series features and have combined them with encrypted features via a Transformer model (XLNet [10]). Prior to feeding the data into the model, we have encoded all features numerically to improve efficiency and reduce overhead during training and testing.
- (3) We have proposed CNNRes-DIndRNN, a novel hybrid module that combines 1D-CNN and RNN to jointly capture local patterns and temporal dependencies. It has achieved high classification accuracy with improved training and testing efficiency.

2. Related Work

2.1. TLS Protocol-Based Network Traffic Classification

Transport Layer Security (TLS) is a cryptographic protocol that secures network communication and serves as an upgraded version of SSL. Compared to SSL, TLS introduces improvements in three key areas: handshake mechanisms, cipher suites, and alert protocols. TLS ensures confidentiality, authentication, and data integrity by combining symmetric (e.g., AES, DES), asymmetric (e.g., RSA, ECC), and hashing algorithms (e.g., SHA-256). It uses digital certificates for identity verification and key exchange protocols such as RSA or Diffie-Hellman to generate session keys.

In TLS-based communications, the process begins with a handshake to establish a secure session. The handshake involves several steps. First, the client sends a list of supported protocol versions and cipher suites, enabling the server to assess the client's encryption capabilities. Then, the server selects appropriate cryptographic parameters and sends a digital certificate for identity verification. Next, both sides exchange keys via public-key cryptography to derive a shared session key. Finally, encrypted communication begins.

Encrypted network traffic makes data content nearly invisible, rendering traffic classification methods relying on plaintext ineffective. As a result, traditional content-dependent

classification methods struggle to identify encrypted malicious traffic and assess security threats. While encryption enhances data security, it creates challenges for detection systems, reducing their ability to detect malicious activities. Bonfiglio et al. [11] (2007) proposed a method for detecting Skype-encrypted traffic using statistical analysis of bitstream randomness and packet characteristics, such as length and inter-arrival time. However, this method relies on Skype-specific protocol signatures, which makes it prone to protocol updates and reduces its accuracy in TCP/VPN traffic. Korczyński et al. [12] (2014) proposed a Markov chain-based method for SSL/TLS traffic classification that generates probabilistic fingerprints from protocol message sequences. However, its reliance on specific SSL/TLS behaviors requires fingerprint updates whenever application behavior changes. Husák et al. [13] (2016) developed a passive SSL/TLS fingerprinting method for HTTPS client identification. By correlating unencrypted TLS metadata with HTTP User-Agent strings, it achieved 95.4% accuracy. However, ambiguity between fingerprint and User-Agent strings complicates identification. Shen et al. [14] (2017) developed a 2D feature graph from SSL/TLS certificates and application packet lengths to model application attributes, employing a second-order model for encrypted traffic recognition. Feature drift occurs when applications or TLS implementations change, requiring fingerprint updates to sustain classification accuracy. Anderson et al. [15] (2018) analyzed TLS handshake discrepancies and flow dynamics between benign and malicious traffic. They developed a hybrid approach integrating logistic regression and rule-based classification to detect encrypted threats and classify malware families. However, poor performance on specific families (e.g., Dridex) highlighted the limitations of current feature engineering methods.

Shekhawat et al. [16] (2019) used Zeek (developed by the Zeek Project, USA) to extract four-tuple logs (source/destination IP, port, protocol) from raw traffic and derived features like connection statistics, SSL handshake attributes, and X.509 certificate properties. Lucia et al. [17] (2019) used TLS message size and direction as SVM/CNN inputs to model encrypted traffic's spatial semantics, but their method overlooks TLS version differences, potentially affecting detection accuracy. Dai et al. [18] (2019) used Zeek (developed by the Zeek Project, USA) to extract flow statistics, SSL handshake data, and certificate attributes, applying mutual information to select features and improve model performance. Hu et al. [19] (2020) extracted plaintext fields from Client/Server Hello messages as features and used logistic regression for encrypted traffic detection. Ferriyan et al. [20] (2020) proposed TLSVec, which converts Client/Server Hello plaintext fields into Word2Vec embeddings. Experimental results indicate near-perfect detection accuracy ($F1 \approx 1.0$) for encrypted malicious traffic. Brzuska et al. [21] (2022) analyzed TLS 1.3's deployment, focusing on adoption rates, security, performance, and implementation. Their study, based on extensive network data, showed improvements in security and performance over previous versions. However, they found challenges like inconsistent protocol support, poor library compatibility, and regional adoption disparities. Xue et al. [22] (2024) introduced a TLS handshake metadata-driven method for detecting proxy traffic. The method analyzes handshake size, timing, and directional patterns to identify obfuscated traffic. It provides high accuracy, passive monitoring, and protocol-agnostic detection. However, it faces challenges in TLS 1.3, multiplexed traffic, and UDP-based proxy environments.

2.2. Time-Series Algorithm-Based Network Traffic Classification Method

Time-series algorithms analyze the temporal patterns in network traffic data and extract meaningful features for application in various tasks. These algorithms are useful for traffic classification, anomaly detection, prediction, and malware detection. Its key advantages include dynamic dependency, non-invasiveness, and high adaptability. Time-series analysis methods are divided into two categories: one uses image processing on packet

payloads and timestamps, while the other extracts statistical features from packet sizes and timestamps. Draper-Gil et al. [23] (2016) introduced a time-series analysis method leveraging statistical features to enhance classification performance. Key features extracted include flow bytes per second and packet arrival intervals. Experimental results indicate that shorter flow timeouts (e.g., 15 s) generally improve accuracy, whereas longer timeouts benefit specific traffic types, such as VPN-Mail. Vu et al. [24] (2018) introduced an LSTM-based deep learning approach that extracts temporal features from traffic flows to capture long-term dependencies. Shapira et al. [25] (2019) developed FlowPic, a method that converts traffic temporal-size information into image representations for CNN-based classification. However, it only achieved 67.8% accuracy in classifying Tor-encrypted traffic, highlighting challenges with Tor-specific encryption.

Niu et al. [26] (2022) proposed an enhanced LSTM model for APT malware detection by combining time-series and association rule features. The model achieved nearly 100% prediction accuracy and improved classification by 5–10% over traditional methods. The study also found that directly processing raw PCAP files results in inefficient data handling. Tang et al. [27] (2023) introduced a novel method for time-series feature extraction in encrypted application traffic classification. This method focuses on analyzing sequences of null packets to identify critical behavioral traits, enabling effective traffic pattern recognition without traditional payload inspection. Tosef et al. [28] (2023) proposed a SFTS-based traffic classification approach by introducing 69 novel time-series features. Experimental results on 15 public datasets demonstrated up to 5% improvement in classification accuracy on specific tasks. Tosef et al. [9] (2024) proposed NetTiSA, a method that uses packet size time-series analysis to extract 13 core features and adds 7 more for a stronger feature set. Testing on 25 classification tasks, ranging from small networks to 100 Gbps systems, showed NetTiSA performs as well as or better than current methods. However, it slightly lags behind plaintext-based methods in tasks like Tor detection and intrusion detection.

2.3. Network Traffic Classification Based on Feature Encoding

Encoding models are a crucial step in machine learning and deep learning, transforming input data into a format suitable for model processing. These models can be categorized into five types: basic encoding (e.g., one-hot), distributed embedding (e.g., Word2Vec [29], ELMo [30]), deep learning-based (e.g., BERT [31]), sequence feature encoding (e.g., Transformer [32]), and hybrid models (e.g., multimodal fusion). Traditional feature encoding commonly uses one-hot encoding. However, for certain data features, the resulting high dimensionality can hinder preconfigured models from yielding satisfactory results. Therefore, alternative encoding methods are often considered for feature representation. Chen et al. [33] (2019) proposed a CNN-based method for encrypted C&C traffic identification. By converting traffic bytes into numeric vectors via Word2Vec and leveraging server-independent features of malware C&C communication, their multi-window CNN extracts local features and inter-block relationships, achieving 91.07% accuracy in high-precision identification. Li et al. [34] (2020) introduced an HTTP traffic anomaly detection method using weighted Word2Vec segment vectors. It reduces training complexity through TF-IDF-weighted mapping and employs LightGBM-CatBoost algorithms for efficient detection. However, it shows limited adaptability to diverse encryption algorithms and may lack sensitivity to unknown or rare HTTP request patterns.

Ferriyan et al. [20] (2022) proposed TLS2Vec, a Word2Vec-based method for detecting malicious behaviors in encrypted traffic. By analyzing TLS handshake and payload characteristics with LSTM networks, it achieves traffic classification. Experiments show TLS2Vec attains 99.9% detection accuracy, outperforming non-Word2Vec methods. However, performance degrades on imbalanced classes and discretized payload lengths. Kholgh et al. [35]

(2023) developed PAC-GPT, a GPT-3-based framework for generating synthetic network traffic. It includes traffic and packet generators capable of producing both normal and malicious traffic scenarios. Current limitations include support only for ICMP/DNS protocols and poor performance on complex protocols. Tang et al. [27] (2023) introduced an Elmo-encoding and LSTM-based method for encrypted traffic classification. Replacing traditional one-hot encoding, Elmo maps words to fixed-length vectors and generates context-aware dynamic embeddings. Ali et al. [36] (2024) leveraged BERT's semantic feature extraction with MLP for efficient classification of imbalanced network traffic in intrusion detection. Dong et al. [37] (2024) reviewed the significance of pre-training (e.g., BERT, GPT, XLNet) in encrypted traffic analysis. Their self-supervised learning and Transformer architectures enable robust feature extraction, particularly BERT's bidirectional encoder capturing complex traffic patterns.

2.4. Deep Learning-Based Method for Network Traffic Classification

With advancements in deep learning, researchers have increasingly explored its application to encrypted traffic identification. For instance, Wang et al. [38] (2017) proposed an end-to-end 1D-CNN method for encrypted traffic classification. The experimental results of the ISCX VPN-NonVPN dataset show that 1D-CNN outperforms the existing methods in multiple evaluation metrics, especially in VPN traffic classification. However, they also show that the performance of 1D-CNN in non-VPN traffic classification needs to be improved. Wang et al. [39] (2017) introduced a 2D-CNN malicious traffic classification method. The method maps the raw traffic to grayscale images and converts them to IDX format as model input. Validation on the USTC-TF2016 dataset demonstrated an accuracy of 99.41%. The advantage of this method is that it can directly process the raw data, thereby reducing the reliance on manual feature engineering. Wu et al. [40] (2018) proposed Bot-Catcher, a CNN-LSTM hybrid system for extracting spatio-temporal features of botnets. While effective, the structural complexity of the model (3M parameters) requires extended training and inference time and significant computational resources. Lotfollahi et al. [41] (2020) developed a CNN-autoencoder framework for encrypted traffic classification. The experimental results show that it achieved 0.98 and 0.94 recall for application and service identification. However, preprocessing steps like Ethernet header removal and traffic filtering add to the implementation complexity.

Aceto et al. [42] (2021) proposed a multimodal multi-task DL method for cryptographic traffic classification and named it Distiller. According to the evaluation, the average accuracy and F1 score were improved by 8.45% and the training time was reduced by 41.7% as compared to existing models. Li et al. [43] (2022) designed a lightweight CNN-SIndRNN architecture for malicious TLS detection. While effectively capturing local patterns and long-range dependencies via 1D-CNN and enhanced SIndRNN, the method exhibits degraded performance on low-sample malware families (e.g., WannaCry), necessitating improved generalization. Bader et al. [44] (2022) proposed an improved DISTILLER-based model (MalDIST) that incorporates statistical features of data packets to enhance feature learning. This method was successfully applied to the detection and classification of malicious encrypted traffic, achieving 99.7% accuracy, precision, recall, and F1 score. Shekhawat et al. [16] (2023) systematically compared the performance of SVM, Random Forest and XGBoost in encrypted traffic analysis. Although RF and XGBoost are close to perfect in terms of accuracy ($\approx 99\%$), their selected features are inferior to SVM in terms of interpretability and intuitiveness. Huo et al. [45] (2023) proposed a multi-view collaborative classification model (MCC) based on semi-supervised learning. The model uses stream metadata features and TLS certificate features to build XGBoost and random forest classifiers. By employing a co-training strategy, it effectively enhances the detection of malicious behavior in encrypted

traffic. Zhao et al. [46] (2024) propose a graph representation-based method for malicious TLS traffic detection (GCN-RF). By converting network traffic into graph structures and leveraging GCNs, the method improves detection performance. However, the use of GNNs also increases the model complexity, resulting in longer training and inference times and higher computational cost. Guo et al. [47] (2024) proposed an encrypted traffic classification method based on low-dimensional second-order Markov matrix (LDSM). By constructing a state transition matrix and using Gini gain for feature dimension reduction, the model complexity and computational overhead are reduced, and the classification efficiency and accuracy are improved.

In conclusion, this study combined SSL/TLS encryption features, time-series analysis, encoding techniques and deep learning to propose a more accurate and efficient encrypted malicious traffic detection method. We proposed using both encryption features and time-series features in our experiments. In order to effectively capture the temporal dependency in traffic data, we employed a transformer-based XLNet encoding model for feature representation, which enhances the model's ability to understand sequence patterns. By integrating a convolutional neural network (CNN) and a recurrent neural network (RNN), the model extracts local temporal features and captures long-term dependencies, thereby improving detection accuracy and optimizing training and testing efficiency.

3. System Architecture and Design

3.1. Overall Workflow

The proposed method for identifying TLS-encrypted malicious traffic consists of four phases, as illustrated in Figure 1.

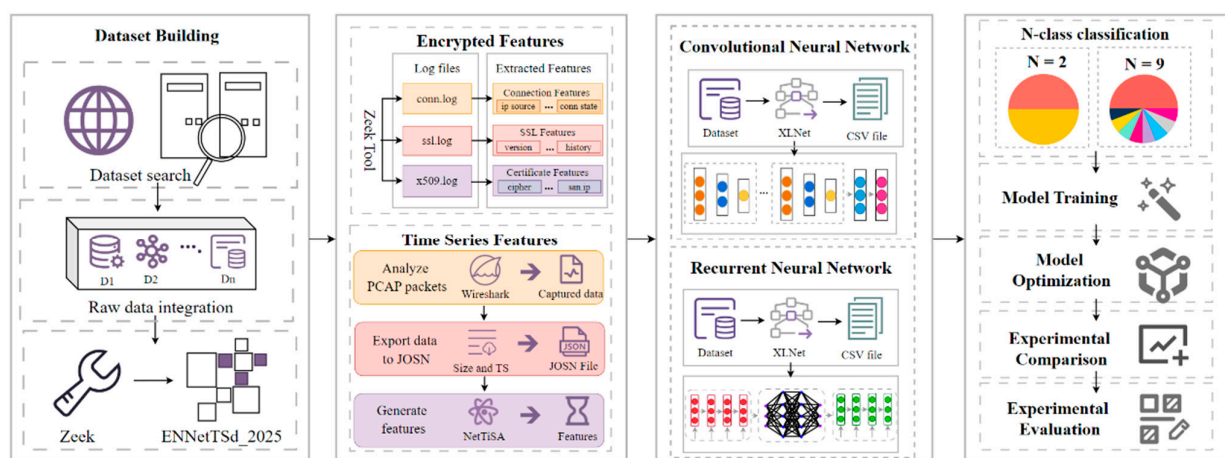


Figure 1. Workflow of CNNRes-DIndRNN-based framework.

The first phase involves constructing the ENNetTSd_2025 dataset. We collected more than ten publicly available malicious traffic datasets from the Internet, including CIC-2017, CIC-2018 [48] and KDD [49]. These datasets were filtered using the Zeek tool (developed by the Zeek Project, USA) based on predefined criteria to extract TLS-encrypted PCAP files. Same-category traffic files from different datasets were merged into the final ENNetTSd_2025 dataset.

The second phase is data cleaning. The encrypted traffic logs filtered by the Zeek tool (developed by the Zeek Project, USA) were merged based on timestamps (TS), five-tuple metadata (source and destination IPs, ports, transport protocol), and unique identifiers (UID). Then, Wireshark was used to extract five-tuple metadata, timestamps and payload sizes from PCAP files to generate structured JSON output. Next, the NetTISA time-series algorithm was applied to process the JSON files and encryption features were extracted

from logs. Finally, the time-series and encryption features were combined to construct the model's input feature set.

The third phase consists of data encoding and model construction. It begins with selecting appropriate encoding models. Several encoding models (e.g., BERT, ALBERT) were used to encode the dataset. The resulting data was then fed into different models for experimental comparison to select the optimal encoder. In this study, the Transformer-based encoder XLNet was chosen to capture contextual dependencies and generate a new CSV file. Subsequently, the model was constructed based on the experimental results. A CNN module and an RNN module were integrated to form CNNRes-DIndRNN. Finally, the generated CSV file was input into CNNRes-DIndRNN.

The fourth phase involves experimental evaluation and comparative analysis. CNNRes-DIndRNN was trained and optimized, and its effectiveness was evaluated by tuning the parameters during the training process. After finalizing the model architecture and parameters, CNNRes-DIndRNN performed binary classification experiments and multi-class comparisons with different models. Finally, the overall performance of CNNRes-DIndRNN was evaluated by various results.

3.2. ENNetTSd_2025 Dataset

To evaluate the effectiveness of the CNNRes-DIndRNN based encrypted malicious traffic classification method, this experiment uses an encrypted malicious traffic dataset. Publicly available and complete TLS-based malicious traffic datasets are currently lacking. Therefore, the Zeek tool (developed by the Zeek Project, USA) was used to filter PCAP files from multiple public traffic datasets. The filtering criterion was whether the files had successfully generated X.509 certificate logs and SSL logs after being processed by the Zeek tool (developed by the Zeek Project, USA). The screening results were malicious traffic data using SSL/TLS encryption protocols obtained from various public traffic datasets. After data screening, the finalized datasets are listed in Table 1. The datasets were divided into two parts: the first contained benign traffic from CIC-IDS2017 [3], MCFP-Normal [4] and VPN-NonVPN [6]; the second consisted of malicious traffic from MCFP [4], CTU-13 [5], and USTC-TF2016 [7].

Table 1. Dataset Sources.

Traffic	Data Sources	Type	Traffic	Data Sources	Type
Benign	MCFP-Normal	Benign	Neris	CTU-13	Malicious
Benign	CIC-IDS2017	Benign	Neris	USTC-TFC2016	Malicious
Benign	VPN-NonVPN	Benign	Razy	MCFP	Malicious
Artemis	MCFP	Malicious	TrickBot	MCFP	Malicious
Bunitu	MCFP	Malicious	Zeus	USTC-TFC2016	Malicious
Dridex	MCFP	Malicious	Zeus	MCFP	Malicious
Htbot	MCFP	Malicious	Neris	CTU-13	Malicious
Htbot	USTC-TFC2016	Malicious	Neris	USTC-TFC2016	Malicious

3.3. Data Processing

The data processing consisted of four consecutive steps. Their purpose was to convert the original labeled PCAP files into input features required by the model. These steps include traffic extraction, feature calculation, feature selection, and feature fusion.

Step I: TLS-encrypted PCAP files are processed using Wireshark's Tshark tool to extract 12 key raw features. These features include five-tuple metadata (source IP, destination IP, source port, destination port, transport layer protocol), packet-level timestamps, and payload size (see Table 2 for details). These raw features will form the basis for subsequent time-series feature calculations. The extracted results will be saved in JSON format. During

this process, the first timestamp extracted from the packet will be used as the data stream timestamp and recorded in UNIX time format.

Table 2. Metadata Fields Extracted from PCAP Files Using Tshark.

Feature Name	Description	Feature Name	Description
TS	Timestamp	SCR_IP	Source IP Address
SCR_Port	Source Port	DST_IP	Destination IP Address
DST_Port	Destination Port	Proto	Protocol
Packet_Count	Total Packet Count	Payload_Sizes	Total Payload Size
P_Timestamps	Packet Timestamps	R_P_Sizes	Reverse Flow Payload Size
R_P_Count	Reverse Flow Packet Count	R_P_Timestamps	Reverse Flow Timestamps

Step II: The JSON file is processed using the NetTiSa algorithm to extract time-series features. This algorithm has been extensively validated, showcasing its lightweight nature, versatility, and high performance in network traffic classification tasks. As a result, it was chosen for time-series feature computation. The specifics of the extracted features are shown in Table 3.

Table 3. Time-series Features.

Time-Series Feature	Description
Packets	Total number of packets for the flow
Bytes	Total number of bytes transferred by the flow
Max.Payload	Maximum packet payload in a flow
Rms.Payload	Root mean square value of the load
Avg.Dispersion	The degree of dispersion of packet interarrival times
Std.Payload	The standard deviation of the packet payload
Variance.payload	Variance of packet payloads
Burstiness	Measure the burstiness of traffic
Percent.Deviation	The degree of deviation from packet load variation
Time.Distribution	Distribution of packet interarrival times
Mean.Payload	Average load per packet
Coefficient.Variation	Ratio of load standard deviation to mean
Reverse.Packets	Total number of reverse packets
Reverse.Bytes	The total number of bytes transmitted by the stream in the reverse direction
Min.Payload	The minimum data packet payload size in the stream
Max.Minus.Min	The difference between the maximum and minimum value of the time interval
Kurtosis.Payload	Kurtosis of the packet payload size distribution
Mean.Relative.Times	Relative mean of packet interarrival times in a flow
Mean.Time.Diff	The average of the inter-arrival times of adjacent packets
Min.Time.Diff	The shortest time interval between the arrival of adjacent packets in a flow
Max.Time.Diff	The longest time interval between the arrival of adjacent packets in a flow
Direction.Ratio	Ratio of the number of forward and reverse packets
Duration	The duration of the flow from the first to the last packet
Switching.Ratio	The ratio of the number of direction changes in the flow to the total number of packets

Step III: Processing Conn.log, SSL.log, and X509.log generated by Zeek (developed by the Zeek Project, USA). According to the “Resumed” field (T/F) in the SSL.log, the fingerprint information is replicated by matching the most recent data flow with the same five-tuple. Then, the information in the X509 certificate is merged with the corresponding content in the SSL log. During the merging process, priority is given to determining whether the certificate is a server certificate. Finally, the merged certificate is integrated with Conn.log, and the features related to encrypted traffic are filtered and extracted.

The selected encryption features are based on key parameters such as fingerprint information, server certificate content, SSL/TLS certificate versions, and other relevant factors. These features enable accurate identification of encrypted traffic and support the detection of malicious activity. The final set of selected encryption features is shown in Table 4.

Table 4. Encrypted Traffic Features.

Encrypted Feature	Description	Encrypted Feature	Description
Cipher	Cipher Suite	State	Current Conn Status.
Cert.fps	Chain Fingerprints	Cert.sig.alg	Signature Algorithm
Server.cert	Server Cert Count	Cert.key.length	Public Key Length
Im.cert	Inter Cert Count	Conn.history	State History of TCP
Root.cert	Root Cert Count	Ssl.history	Conn Status: SSL/TLS
Cert.version	Cert Version	Version	SSL/TLS Version
Cert.serial	Cert Serial Number	Cert.issuer	Cert Issuer
Cert.subject	Cert Subject	Cert.key.alg	Public Key Algorithm
Total.cert	Number Of Cert	Cert.key.type	Public Key Type
Cert.not.valid.before	Validity Start Time	Cert.not.valid.after	Validity Start Time

Step IV: Based on the five-tuple information and timestamps, the encrypted features were matched and integrated with the time-series features. The fused data were then subjected to statistical analysis, ultimately yielding 12 data types (as shown in Figure 2). To ensure the rigor and balance of the experiment, datasets of identical sizes were used for both binary and multi-class classification experiments. Ultimately, nine data categories were selected for experimentation (as shown in Figure 3), with a 7:3 split ratio for training and testing. To validate the effectiveness of the selected features, the SHAP method was employed for feature evaluation. Based on the evaluation results, the final set of required features was determined, as detailed in Table 5.

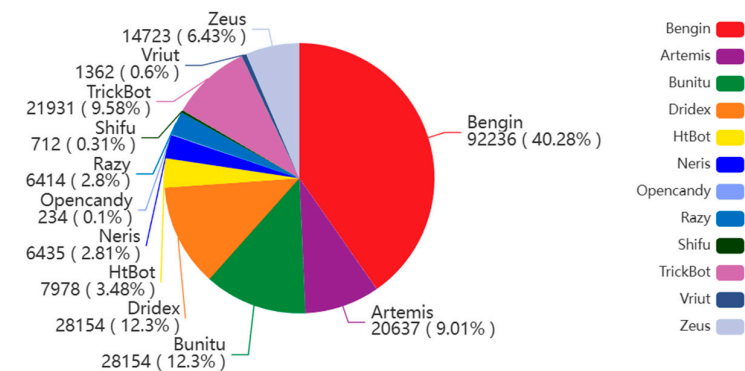


Figure 2. 12 Network Traffic Types with Volumes.

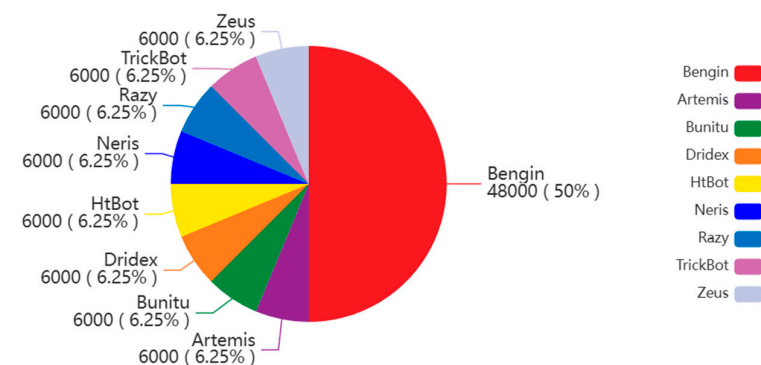


Figure 3. Filtered Types and Traffic Volumes.

Table 5. Features.

Features	Features	Features	Features
Cipher	Cert.chain.fps	Rms.Payload	Max.Minus.Min
Total.cert	Cert.sig.alg	Avg.Dispersion	Kurtosis.Payload
Server.cert	Im.cert	Std.Payload	Mean.Relative.Times
Root.cert	Cert.key.length	Mean.Time.Diff	Burstiness
Cert.subject	Cert.serial	Min.Time.Diff	Percent.Deviation
Cert.not.valid.before	Cert.not.valid.after	Max.Time.Diff	Time.Distribution
Cert.issuer	Version	Switching.Ratio	Mean.Payload
Ssl.history	Conn.history	Direction.Ratio	Dratation
State	Max.Payload	Packets	Bytes
Reverse.Bytes	Reverse.Packets		

3.4. Feature Encoding

In the encoding selection phase, the experiment first analyzed and compared common word vector representations, and then discussed the advantages of dynamic embedding models. Based on the experimental results, the XLNet model was ultimately chosen as the encoding model. Traditional approaches (e.g., one-hot encoding) suffer from high-dimensional sparsity and an inability to capture semantic relationships, and thus are not suitable for our dataset. Although static embeddings (e.g., Word2Vec) can partially capture semantic similarity, they fail to model context dependencies in encrypted malicious traffic due to static representation. In contrast, context-sensitive dynamic embedding-based models (e.g., XLNet [10], BERT [31], RoBERTa [50], and ALBERT [51]) can dynamically generate word representations based on varying contexts, thereby better capturing semantic diversity and dependencies. Therefore, experiments were conducted using these models, and the final selection was made based on the experimental results, as shown in Table 6. Details of the SVM, XGBoost, and RF models we used are provided in Section 4.3.

Table 6. Post-Encoding Experimental Results.

Encoding Methods	SVM [16]	XGBoost [16]	RF [16]
XLNet [10]	99.11%	97.89%	87.84%
BERT [31]	98.91%	97.63%	85.20%
RoBERTa [50]	97.39%	95.90%	84.40%
ALBERT [51]	97.17%	96.07%	83.40%

XLNet

The core innovation of XLNet lies in its Permutation Language Modeling (PLM) framework. This mechanism generates diverse contextual combinations by dynamically rearranging token sequences. In doing so, it overcomes the limitations of the static masking paradigm used in traditional autoencoder models (e.g., BERT). Specifically, the objective function of XLNet is to maximize the expected log-likelihood over all word order permutations. During training, it randomly samples different permutations in each iteration, enabling the model to adapt to context dependencies in various directions. By aggregating predictions across all permutations, XLNet learns both forward and backward contextual dependencies within a sentence. In contrast, it overcomes the unidirectional limitation of traditional autoregressive models such as GPT, and also avoids the missing training signals

caused by masking words in BERT, thereby improving the efficiency of data utilization. Its formula (1) is as follows:

$$\mathcal{J}_{\text{XLNet}} = \max_{\theta} \mathbb{E}_{z \sim \mathcal{Z}_T} \left[\sum_{t=1}^T \log p_{\theta}(x_{z_t} | x_{z_{<t}}) \right] \quad (1)$$

where z is the random variable for permutation ordering, $\mathcal{Z}_T$ denotes the set of all permutations of a sequence with length T (number of tokens), x_{z_t} represents the word at the t -th position under permutation z , and $x_{z_{<t}}$ indicates the permuted context preceding position t . The model parameter θ is optimized by maximizing the joint probability over all permutations, where $\mathbb{E}_{z \sim \mathcal{Z}_T}$ denotes the expectation across all possible permutations.

In the PLM framework, if we directly apply the self-attention mechanism of the standard Transformer, it will cause the model to leak information when predicting the target word. To address this issue, XLNet proposes Two-Stream Self-Attention to separate the content stream from the query stream. Among them, the content flow allows the encoding to capture the complete context information of the current word x_{z_t} . This enables the construction of a deep semantic representation. When the query stream generates the prediction signal, it masks the current word information and only relies on the legal context x_{z_t} . The formula for the content stream is given in Equation (2), and the formula for the query stream is given in Equation (3):

$$h_{z_t}^{(m)} = \text{Attention}(Q = h_{z_t}^{(m-1)}, KV = h_{z_{\leq t}}^{(m-1)}; \theta) \quad (2)$$

$$g_{z_t}^{(m)} = \text{Attention}(Q = g_{z_t}^{(m-1)}, KV = h_{z_{<t}}^{(m-1)}; \theta) \quad (3)$$

where $h_{z_t}^{(m)}$ denotes the content representation vector of position z_t in the m -th layer, with Q, K and V representing the Query, Key, and Value matrices, respectively. $z_{\leq t}$ indicates the indices of all words up to position t in the permutation, and $h_{z_{\leq t}}^{(m-1)}$ is the set of hidden states for these positions in the $(m-1)$ -th layer. The query representation vector $G_{z_t}^{(m)}$ corresponds to position z_t in the m -th layer, while $z_{<t}$ refers to indices of all preceding words in the permutation. The hidden states $h_{z_{<t}}^{(m-1)}$ from the $(m-1)$ -th layer aggregate information from positions $z_{<t}$, with θ representing shared model parameters.

To enhance sensitivity to relative word positions, XLNet replaces traditional fixed positional encoding with learnable relative position embeddings, enabling flexible modeling of inter-word distance relationships. The formulation of the relative positional encoding is provided in Equation (4):

$$A_{i,j} = q_i^{\top} k_j + q_i^{\top} r_{i-j} + u^{\top} k_j + v^{\top} r_{i-j} \quad (4)$$

where q_i and k_j denote the Query and Key vectors. The relative distance embedding r_{i-j} between positions i and j is a learnable parameter, while u and v are globally learnable bias parameters.

The final prediction utilizes the top-level output of the query stream. Probability computation is achieved through parameter sharing in the word embedding matrix, and its formula is shown in Equation (5):

$$P(x_{z_t} | x_{z_{<t}}) = \text{softmax}(g_{z_t}^{(M)} W_E) \quad (5)$$

where $g_{z_t}^{(M)}$ denotes the output vector of the M -th (top) query stream, W_E represents the word embedding matrix with parameters shared from the input embeddings, and the Softmax function maps the vector to a probability distribution.

After analyzing different embedding methods and experimental results, the XLNet model was chosen for word vector encoding. This module addresses two major issues in traditional static embedding methods: high-dimensional sparsity and context independence. It dynamically adjusts feature representations to better capture the characteristics of encrypted malicious traffic. These include dynamic behaviors and complex relationships in time series. As a result, the detection of encrypted malicious traffic becomes both more accurate and more robust.

3.5. Overall Model Architecture

The experimental model consists of a CNN convolutional neural network module and an RNN convolutional neural network module. According to the experimental results in Tables 7 and 8, 1D-CNN and IndRNN modules were selected. The overall model architecture integrates the advantages of 1D-CNN and IndRNN, which improves the efficiency of local feature extraction in time series while effectively enhancing the modeling capability of temporal dependencies. The Batch Normalization (BN) module and Residual (Res) module used in the model were determined through experiments, based on the CNN experimental results in Table 9, the RNN experimental results in Table 10, and the ablation experiments of the CNN-RNN combination in Figure 6. Based on the comprehensive analysis of these three experiments, the final model architecture is shown in Figure 4.

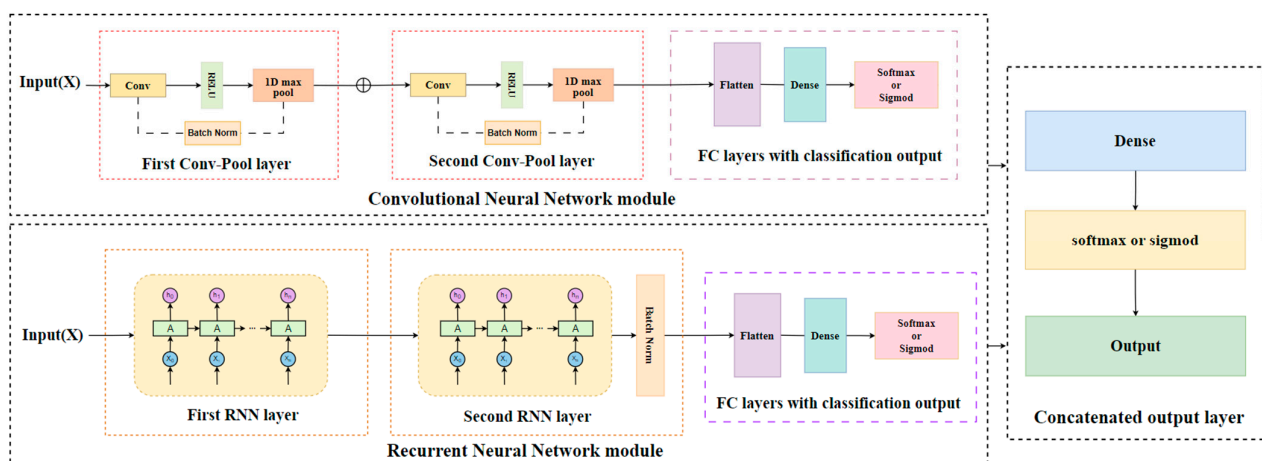


Figure 4. Architectural Framework of the CNNRes-DIndRNN Model.

3.5.1. CNN Module

The convolutional neural network part consists of convolutional layer, pooling layer, residuals, and fully connected layer. This study adopts a 1D-CNN architecture for the experiments. The specific structural functions of the CNN convolutional neural network part are described below:

(1) First Convolutional-Pooling Layer

This layer extracts input features using 128 (3×1) convolutional kernels, generating 128 feature maps. These are then processed with batch normalization and a ReLU activation function. The feature maps are next compressed using a 2×1 max pooling operation. Finally, a residual connection adjusts the input dimensions with a convolution of stride 2.

(2) Second Convolutional-Pooling Layer

This layer is similar to the first layer but with simplified parameters: it uses 64 (3×1) convolution kernels to generate 64 feature maps of size 384×1 . After batch normalization, ReLU activation, and 2×1 max pooling, the feature map size is reduced to 192×1 . No

residual connection is used in this layer, making the structure simpler and helping to reduce model complexity.

(3) Fully Connected Layer

The feature maps are flattened and input into the fully connected layer. The layer contains 64 neurons. A nonlinear transformation is introduced using the ReLU activation function. The risk of overfitting is reduced by Dropout. The final result of the classification task is output through the output layer ($M = 9$ or 1), using Sigmoid (binary classification) or Softmax (multi-classification) activation functions.

3.5.2. RNN Module

IndRNN is selected as the core recurrent unit in this study to improve the sequence feature capture ability. The IndRNN [52] is integrated as a module. The specific structural functions of the RNN recurrent neural network part are described below:

(1) First RNN Layer

This layer consists of 128 IndRNN units with default activation function tanh and ReLU activation function. The `return_sequences` parameter is set to True. The recurrent weights are constrained with a `recurrent_max_abs` value of 2.0. Dropout is added after the IndRNN layer to prevent overfitting, with a dropout rate set to 0.3.

(2) Second RNN Layer

This layer uses 64 IndRNN units, with the recurrent weight constraint tightened to a `recurrent_max_abs` of 1.0 to further enhance training stability. The `return_sequences` parameter is set to False to retain only the output of the final time step. Batch Normalization is applied to the 64-dimensional features output by the IndRNN to normalize them, enhancing training efficiency and stability.

(3) Fully Connected Layer

The feature maps are flattened and input into the fully connected layer. The layer contains 64 neurons. A nonlinear transformation is introduced using the ReLU activation function. The risk of overfitting is reduced by Dropout. The final result of the classification task is output through the output layer ($M = 9$ or 1), using Sigmoid (binary classification) or Softmax (multi-classification) activation functions.

3.5.3. Model Concatenation

The CNN and RNN modules extract features from the input data independently and perform self-learning. Their outputs are concatenated together to obtain a new N ($N = 2, 18$) dimensional vector, which contains the classification results of the traffic types judged by the two models from different perspectives. The concatenated vector is first passed through a fully connected layer with 64 neurons. It is then processed by the ReLU activation function to produce a 64-dimensional feature vector. To avoid overfitting, a dropout layer is used after the fully connected layer. Next, it goes through an output layer that uses an activation function to perform classification. The output is a probability distribution vector of length N ($N = 1$ or 9), representing the predicted probability for each class. Finally, the predicted probability indicates a specific class of traffic.

3.6. Classification Learning

Before performing classification, the results of the convolutional neural network and the partial output of the recurrent neural network need to be aggregated. In this paper, the concatenation method is used to aggregate the output data, and the result is input into the classification function, which uses an activation function to output the final decision. For the binary classification task, the Sigmoid activation function is used. It determines

whether the input data flow is normal encrypted traffic or malicious encrypted traffic. The binary crossentropy loss function `binary_crossentropy` is chosen as the loss function. The function formulas are shown in Equations (6) and (7):

$$\text{sigmoid} : \Theta(x) = \frac{1}{1 + e^{-x}} \quad (6)$$

$$\text{Loss}(y_{\text{pre}}, y_{\text{true}}) = - \sum_j^c y_{\text{true}} \times \ln y_{\text{pre}} \quad (7)$$

where x represents the input value, $\Theta(x)$ represents the probability that the input belongs to a positive category, the output ranges in $(0, 1)$. y_{true} represents the true label, y_{pre} represents the probability predicted by the model, and c represents the number of categories ($c = 2$).

For the multi-classification task, the Softmax function is selected as the activation function for the neurons in the output layer. It maps the outputs of multiple neurons to a range between 0 and 1. These outputs sum to 1 and represent the probability of classification into each class. The multivariate cross-entropy loss function `categorical_crossentropy` is chosen for the loss function. The functional formula is shown in Equations (8) and (9):

$$\text{softmax} : S(y_i) = \frac{e^{y_i}}{\sum_j^c e^{y_j}} \quad (8)$$

$$\text{Loss}(y_{\text{pre}}, y_{\text{true}}) = - [y_{\text{true}} \log_a y_{\text{pre}} + (1 - y_{\text{true}}) \log_a (1 - y_{\text{pre}})] \quad (9)$$

where y_i represents the output of the i -th neuron, and the output of $S(y_i)$ is in the range $(0, 1)$, which represents the probability of belonging to the i th class, and the sum of all class probabilities is 1.

The model is trained using the Adam optimizer to adaptively adjust the learning rate. The partial derivatives are calculated according to the loss function to update the model weights W and bias b . Let the weights and biases of the l -th layer of the network model be $W^{(l)}$ and $b^{(l)}$, respectively, and the parameter updating process is shown in Equations (10) and (11):

$$W^{(l)} \leftarrow W^{(l)} - \eta \frac{\partial \text{Loss}(y_{\text{true}}, y_{\text{pre}})}{\partial W^{(l)}} \quad (10)$$

$$b^{(l)} \leftarrow b^{(l)} - \eta \frac{\partial \text{Loss}(y_{\text{true}}, y_{\text{pre}})}{\partial b^{(l)}} \quad (11)$$

where $W^{(l)}, b^{(l)}$ represent the weight and bias of the l -th, η represents the learning rate, and $\frac{\partial \text{Loss}}{\partial b^{(l)}}$ and $\frac{\partial \text{Loss}}{\partial W^{(l)}}$ represent the gradient of the loss function with respect to the parameters.

4. Experimental Results

4.1. Experimental Environment

The environment used in this experiment (Hebei Province, China): Lenovo SAVIOR Y7000P CPU Intel® Core™ i7-9750H CPU @ 2.60 GHZ Graphics1 Intel® UHD Graphics 630 Graphics2 NVIDIA GeForce GTX 1660 Ti.

4.2. SHAP-Based Feature Significance Validation

In order to verify the actual effectiveness of these features for the model, we employed the SHAP [53] method to analyze the contribution of encrypted features and time-series features. The model used in the experiment is XGBoost [54], implemented through the official Scikit-learn library [55] and the official XGBoost library [56]. The number of iterations is set to 100, the learning rate is set to 0.3, and the maximum depth of a single tree is set to 6. Figure 5 shows the distribution of SHAP values. From the figure, it can be seen that among

the 44 input features, the SHAP values of 38 features generally deviate from zero, while the SHAP values of 6 features equal zero. Therefore, based on the experimental results, 38 features listed in Table 5 were selected as inputs for the model experiments.

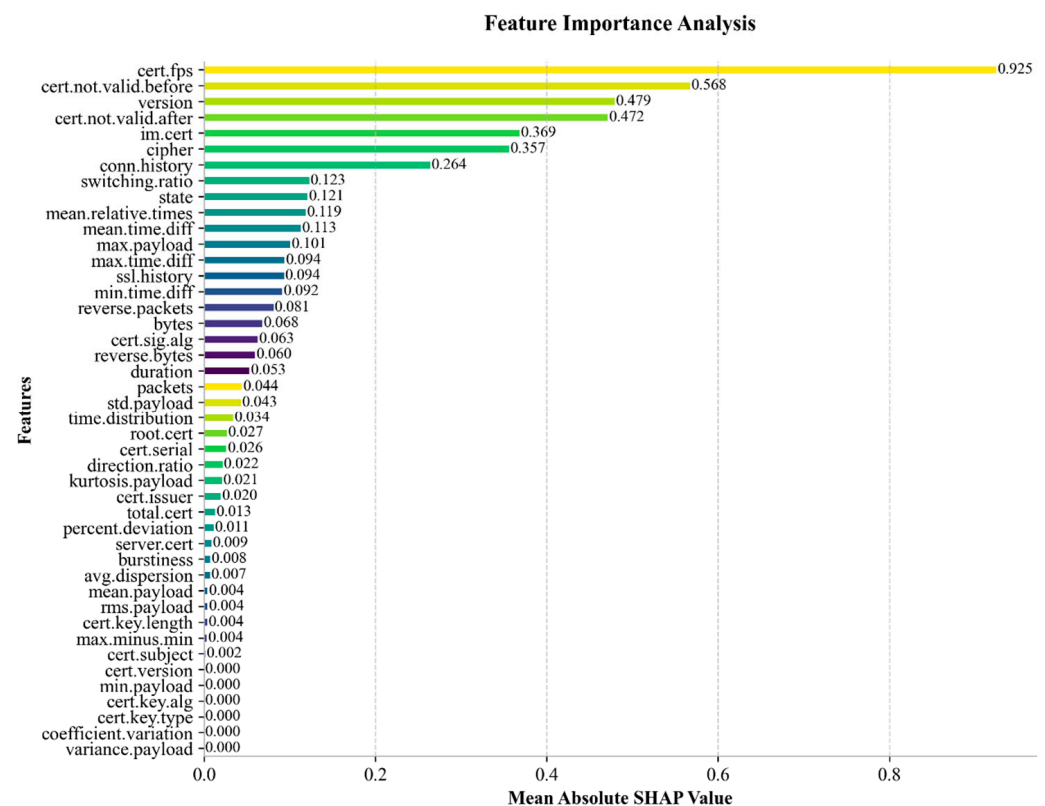


Figure 5. Feature shap contribution value.

4.3. Encoding Method Comparison Experiment

To capture the dependencies in experimental data and select the most suitable encoding model, this paper explores multiple encoding approaches. Specifically, four models—XLNet [10], BERT [31], RoBERTa [50], and ALBERT [51] are used to encode the same feature data for comparative analysis.

All pre-trained encoder models and tokenizers are sourced from the Hugging Face library [57], and no fine-tuning is applied to the encoder models during this process. For example, with the XLNet encoder model, the 38 filtered features are concatenated into a single long text sequence during encoding. Before input, the XLNetTokenizer is used for tokenization, applying auto-completion and truncation to meet the model's input requirements. Finally, the processed sequence is fed into the XLNet-Base-Cased [58] model, generating a unified 768-dimensional vector.

The encoding models were all tested with a dimension of $N = 768$, and the accuracy results were obtained and compared using three models: SVM [16], XGBoost [16] and RF [16]. The SVM model uses a linear kernel function, and the regularization parameter C is set to 0.001. In the XGBoost model, the number of iterations is set to 100, the learning rate is set to 0.01, and the maximum depth of a single tree is set to 6. In the RF model, the number of decision trees is set to 100, and the minimum number of samples required to split a node is set to 2. Finally, the results are shown in Table 6, and the XLNet encoding model was selected for data encoding in this experiment.

4.4. Ablation Experiments and Result Analysis

4.4.1. Ablation Experiments

To validate the rationality of the design of each module of the CNNRes-DIndRNN model, ablation experiments were conducted under the same conditions using the EN-NetTSd_2025 dataset.

Since the data is in the form of one dimension, 1D-CNN is chosen in the convolution module. In the Recurrent Neural Network module, in order to select a suitable RNN model. In this paper, six models including GRU [59], LSTM [60], RNN [61], Bi-GRU [62], Bi-LSTM [63], and IndRNN [64] are, respectively, selected for the experiments. The nine-class classification task is conducted under the experimental condition of Epoch = 1182 on the training set. The performance is compared in terms of training time, testing time and model accuracy. All six RNN models use a two-layer structure. The number of units in the first layer is set to 128 and the second layer is set to 64. The experimental results are summarized in Table 7. The data in the table show that Bi-GRU is 0.12 s faster than IndRNN in test time, and it performs worse than IndRNN in accuracy and training time. Compared with GRU, LSTM, RNN, and Bi-LSTM, IndRNN performs the best in all three indicators.

Table 7. Experimental results of RNN model comparison.

Module	Accuracy	Train Time (S)	Test Time (S)
GRU [59]	97.94%	4.071	1.34
LSTM [60]	90.72%	4.269	1.05
RNN [61]	86.55%	2.473	1.23
Bi-GRU [62]	92.78%	7.270	0.97
Bi-LSTM [63]	92.84%	9.759	2.23
IndRNN [64]	98.77%	2.169	1.09

To determine the type of RNN module to use, we conducted further ablation experiments. These experiments also aimed to verify whether using two modules in a parallel structure outperforms using either module alone. The experimental results are shown in Table 8. A model combining 1D-CNN and RNN in a parallel fusion structure was constructed and compared in terms of accuracy, parameter count, model size, training time, and test time. The CNN module used is a 1D-CNN, with the first layer having 128 convolution kernels of size 3, and the second layer having 64 convolution kernels of size 3. Both layers adopt ReLU activation and max pooling with a window size of 2. The RNN models used are the six models listed in Table 7.

By comparing the results in Tables 7–9, the CNN-IndRNN model achieves an accuracy of 99.04%. This is higher than the accuracy of the IndRNN model (98.77%) and the CNN model (95.49%). It can be seen that the CNN-IndRNN model outperforms both the IndRNN and CNN models. Therefore, it is proved that using both CNN and IndRNN modules simultaneously is superior to using either the IndRNN or CNN module alone.

In Table 8, the CNN-GRU and CNN-BiLSTM models achieve slightly higher accuracies than the CNN-IndRNN model, both reaching 99.06%. In comparison, the CNN-IndRNN model achieves an accuracy of 99.04%. However, in terms of parameter count, model size, training time, and test time, the CNN-IndRNN model performs best, with results of 92,565, 10,945.42 KB, 17.537 s, and 2.948 s. Therefore, the CNN-IndRNN model is finally selected for the next stage of the experiment.

In order to enhance the stability of model training, accelerate convergence, reduce overfitting, and strengthen feature representation, this experiment introduces Batch Normalization (BN) and Residual (Res) modules. First, ablation experiments were performed

on the CNN module. The CNN module used in this experiment is a 1D-CNN, with the first layer having 128 convolution kernels of size 3 and the second layer having 64 convolution kernels of size 3. Both layers adopt ReLU activation and max pooling with a window size of 2. BN blocks and Res blocks were added separately to each layer for experimentation. The positions for BN and Res insertion were determined based on experimental results. Comparisons were made under the same experimental conditions, and the specific results are shown in Table 9. Based on the criteria of accuracy above 98% and loss function less than 0.06, numbers C10, C12, C15, and C16 were finally selected for the next stage of experiments.

Table 8. CNN-R structure comparison experiment results.

Module	Accuracy	Parameter	Size (KB)	Train (S)	Test (S)
CNN-GRU	99.06%	1,196,699	14,115.62	19.927	2.975
CNN-LSTM	94.70%	1,323,291	15,599.13	22.240	2.959
CNN-RNN	89.07%	941,787	10,960.35	20.800	4.211
CNN-BiGRU	98.95%	1,628,507	19,195.66	22.550	3.210
CNN-BiLSTM	99.06%	1,898,075	22,354.69	23.551	3.181
CNN-IndRNN	99.04%	925,659	10,945.42	17.537	2.948

Table 9. CNN module experimental comparison.

Number	1st Layer Config	2nd Layer Config	Accuracy	Loss
C1	None	None	95.49%	0.1592
C2	None	Res	97.12%	0.1056
C3	None	BN	98.01%	0.0653
C4	None	Res + BN	98.21%	0.0600
C5	Res	None	97.07%	0.1026
C6	Res	Res	97.60%	0.0780
C7	Res	BN	98.29%	0.0597

The RNN module used is the IndRNN module, with 128 hidden units in the first layer and 64 in the second layer. Batch Normalization (BN) was introduced in this experiment. The specific results are shown in Table 10. According to the criteria of accuracy above 99% and loss rate below 0.025, models represented by R2, R3, and R4 were finally selected for the next stage of the experiment.

Table 10. RNN module experiment comparison.

Number	1st Layer Config	2nd Layer Config	Accuracy	Loss
R1	None	None	98.77%	0.0462
R2	None	BN	99.43%	0.0214
R3	BN	None	99.31%	0.0241
R4	BN	BN	99.38%	0.0220

This experiment combined numbers C10, C12, C15, and C16 with R2, R3, and R4 to conduct parallel CNN and RNN ablation experiments on the ENNetTSd-2025. According to the experimental results shown in Figure 6, the combination of C10 and R2 achieved the best accuracy, reaching 99.67%. Compared with the CNN-RNN combinations without the Res and BN modules in Table 8, which achieved an accuracy of 99.04%, the combination with these modules improved accuracy by 0.63%. This demonstrates that the CNN-RNN combination using both Res and BN modules performs better. Therefore, the experimental structure finally selected the configuration represented by C15 and R2 for comparative experiments with other methods to verify the effectiveness of this approach.

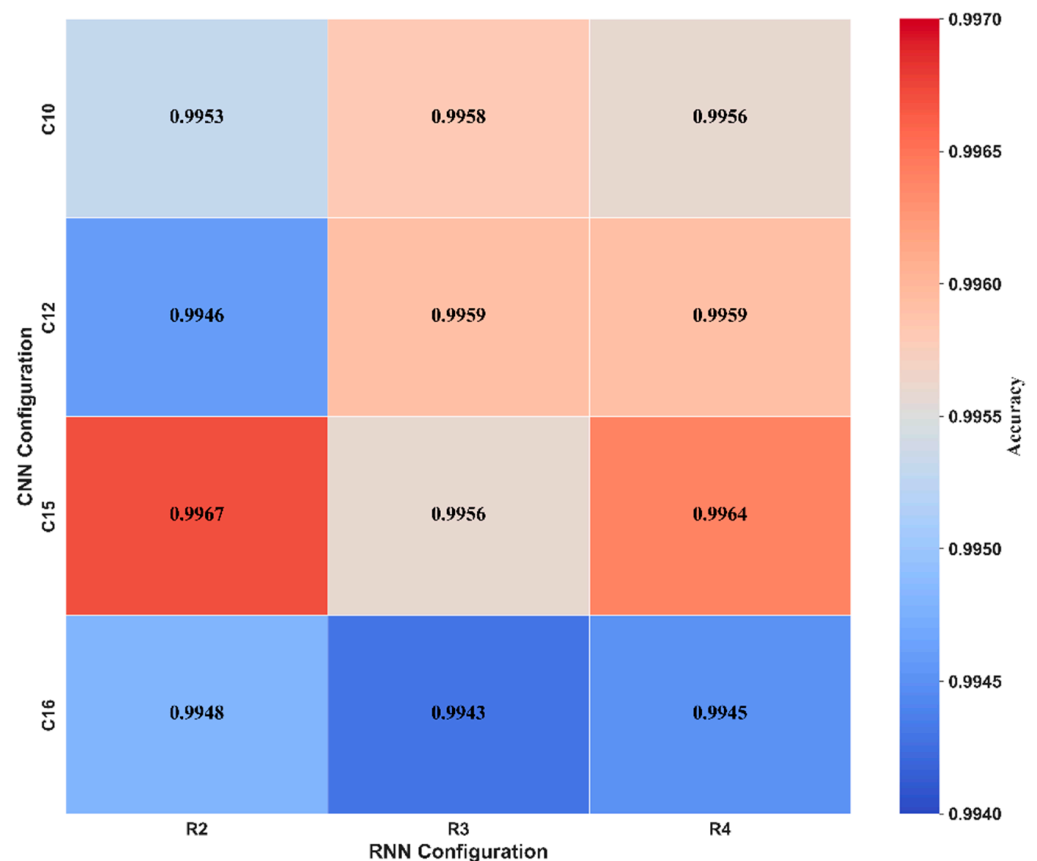


Figure 6. Experimental configuration heat map.

4.4.2. Binary Classification Results Comparison

To validate the effectiveness of the CNNRes-DIndRNN model in recognizing malicious traffic data based on the SSL/TLS encryption protocol, this paper conducts an experimental comparative study on the binary classification of encrypted malicious traffic. The study uses the MCFP, CTU-13, VPN-NonVPN, and CIC-DIS2017 datasets, all of which have been screened to include data with TLS/SSL encryption protocols and encoded with the XLNet model.

SVM [16], Random Forest (RF) [16] and XGBoost [16] models are used to filter HTTPS traffic using the Bro IDS (Zeek, developed by the Zeek Project, USA) [8] tool. Their models construct classifiers based on a total of 38 features. These features are drawn from three dimensions: TCP, UDP connections, TLS, and X509 certificate logs. The paper confirms that different kernel functions have little effect, but linear merging is more suitable for feature analysis. Therefore, the SVM kernel functions used in the experiment is linear. The RF base classifier $n_estimators$ is set to 500, and the Gini Impurity is used as the classification index. In the XGBoost model, the learning rate is set to the default value of 0.1, and the maximum depth is set to the default value of 6.

For 1D-CNN [38] and 2D-CNN [39] experiments, PCAP files were preprocessed by extracting the first 784 bytes of each TLS session stream as input. In the experiments, the 1D-CNN model employs two layers of one-dimensional convolutional networks, while the 2D-CNN model utilizes two layers of two-dimensional convolutional networks. Both models independently extract and learn data features. In the experiments with the BotCatcher [40] model, TLS session streams and packets were preprocessed separately. First, the first 1024 bytes from each TLS session stream were intercepted as input features. Second, the first 10 packets in each connection were extracted (excluding the first two associated DNS

packets), and the first 80 bytes from each packet were intercepted for modeling. The processed data were fed into CNN and LSTM networks for training and testing.

In the CNN_SIndRNN [43] experiments, the method generates an $n \times k$ matrix ($n = 121$, maximum session length; $k = 11$, embedding layer vector dimension). The TLS session flow with a sequence length of 1331 is then segmented for subsequent computations. The DISTILLER [42] model selects a biflow and extracts the first 784 bytes of transport layer payload data as input features. Additionally, it extracts the protocol header fields from the first 32 data packets, reserving four key fields from each packet as input features. These features are then fed into the BiGRU component for time series modeling. The MaIDIST [44] model takes biflow as the basic unit. It extracts 784 bytes of payload data as input to the 1D-CNN, and the protocol header fields from the first 32 packets are input to the BiGRU. On the basis of these inputs, the packets of 14 feature counts are grouped to form a 5×14 data matrix and transmitted to BiLSTM and 2D-CNN.

The experimental results are shown in Table 11, where the bold data indicates the maximum value of the data in this group. As shown in the table, the recall rates of the 1D-CNN and 2D-CNN models are 99.99% and 99.98%, and both models are better than the recall rate of CNNRes-DIndRNN. However, CNNRes-DIndRNN achieves higher accuracy, precision and F1 score than the other 9 models. Therefore, in a comprehensive view CNNRes-DIndRNN is the most effective in detecting encrypted malicious traffic.

Table 11. Binary Classification Benchmark of 10 Models.

Model	Accuracy	Precision	Recall	F1-Score
SVM [16]	96.86%	96.12%	97.69%	96.90%
RF [16]	92.03%	97.39%	86.46%	91.60%
XGBoost [16]	95.27%	94.90%	95.75%	95.32%
1D-CNN [38]	96.56%	95.43%	99.99%	97.66%
2D-CNN [39]	96.42%	95.26%	99.98%	97.61%
BotCather [40]	92.30%	94.78%	88.44%	91.50%
CNN-SIndRNN [43]	95.12%	97.68%	92.39%	98.94%
DISTILLER [42]	97.71%	97.46%	97.94%	97.70%
MaIDIST [44]	99.31%	99.31%	99.30%	99.30%
CNNRes-DIndRNN	99.81%	99.75%	99.87%	99.81%

To objectively evaluate the comprehensive performance of the proposed model in binary classification tasks, ROC curves were plotted based on the distribution characteristics of the False Positive Rate (FPR) and True Positive Rate (TPR), as shown in Figure 7. The experimental results show that the area under the ROC curve (AUC) of the CNNRes-DIndRNN model proposed in this study reaches 0.9999, which is 0.35–3.60% higher than that of the other nine comparison models. Furthermore, its ROC curve is closest to the ideal point (0, 1) in the upper-left corner of the coordinate plane. These findings indicate that the model can effectively suppress the false-positive rate while maintaining a high true-positive rate, and exhibits the most excellent classification performance and generalization ability in the binary classification problem.

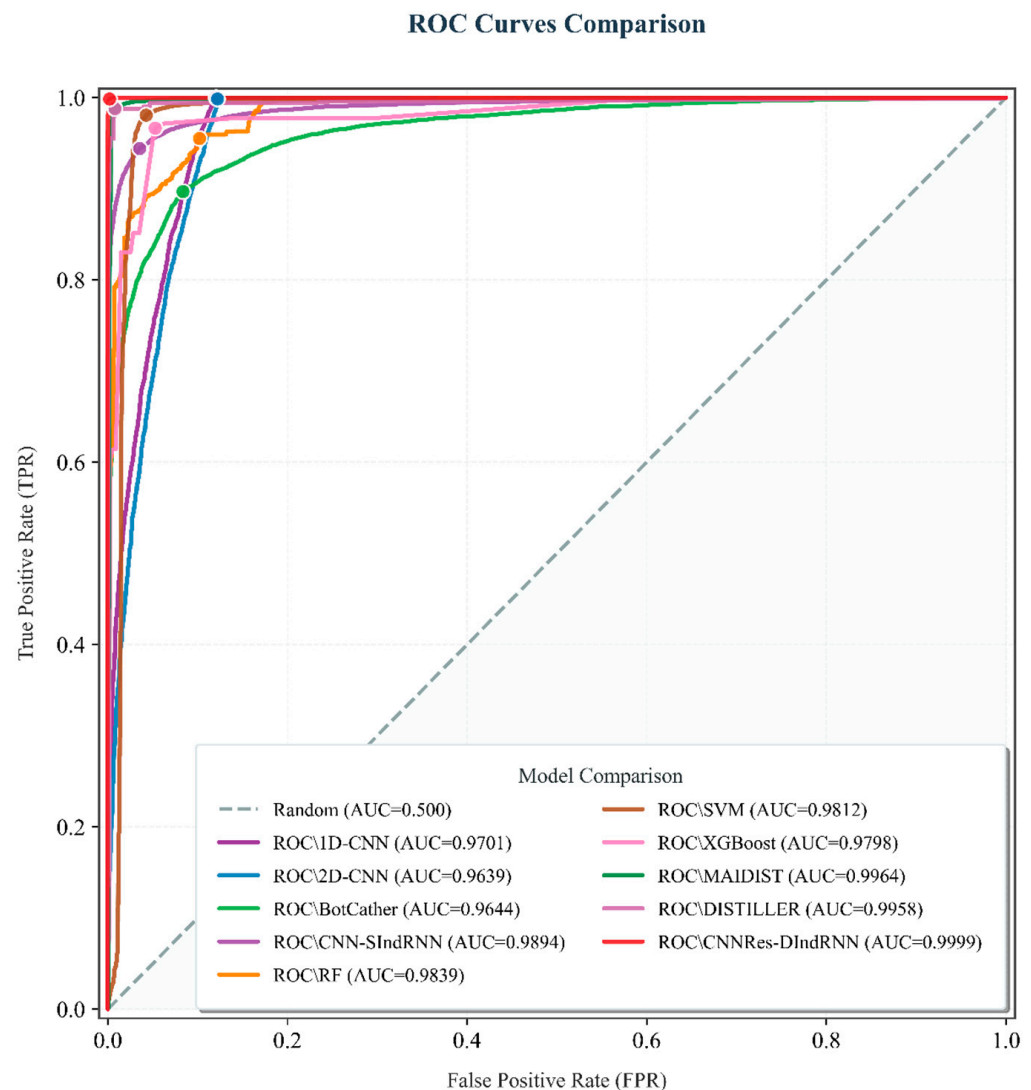
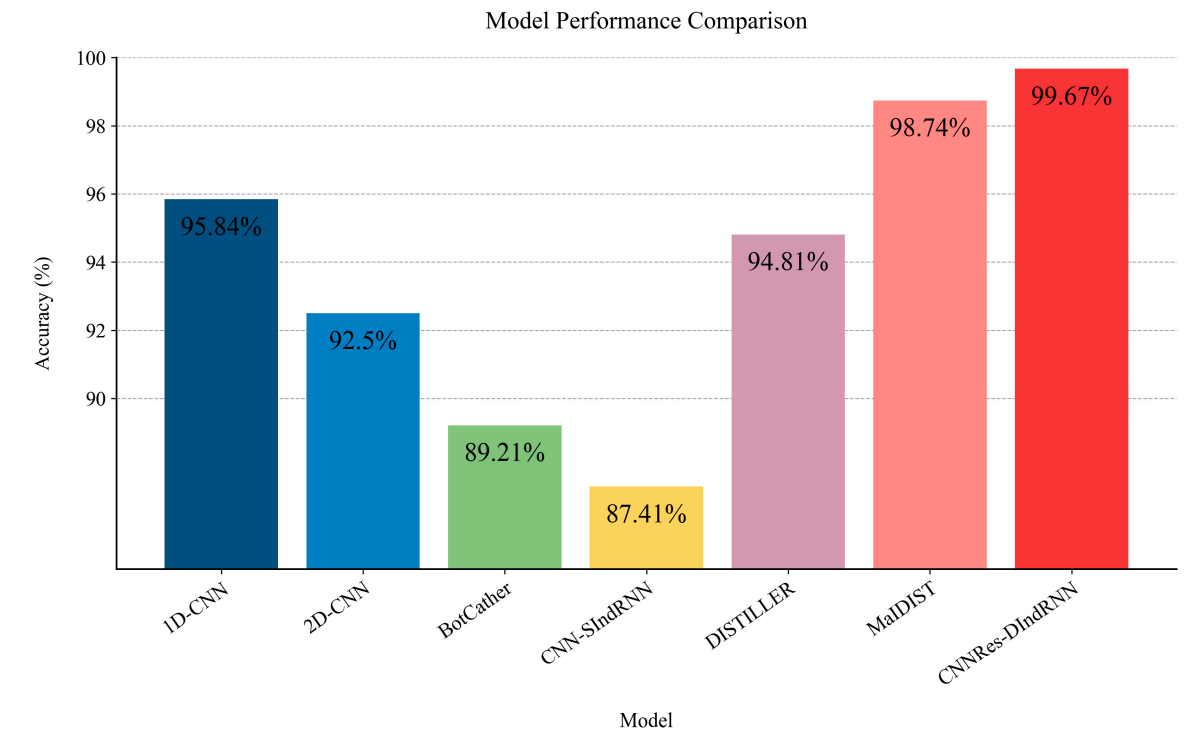
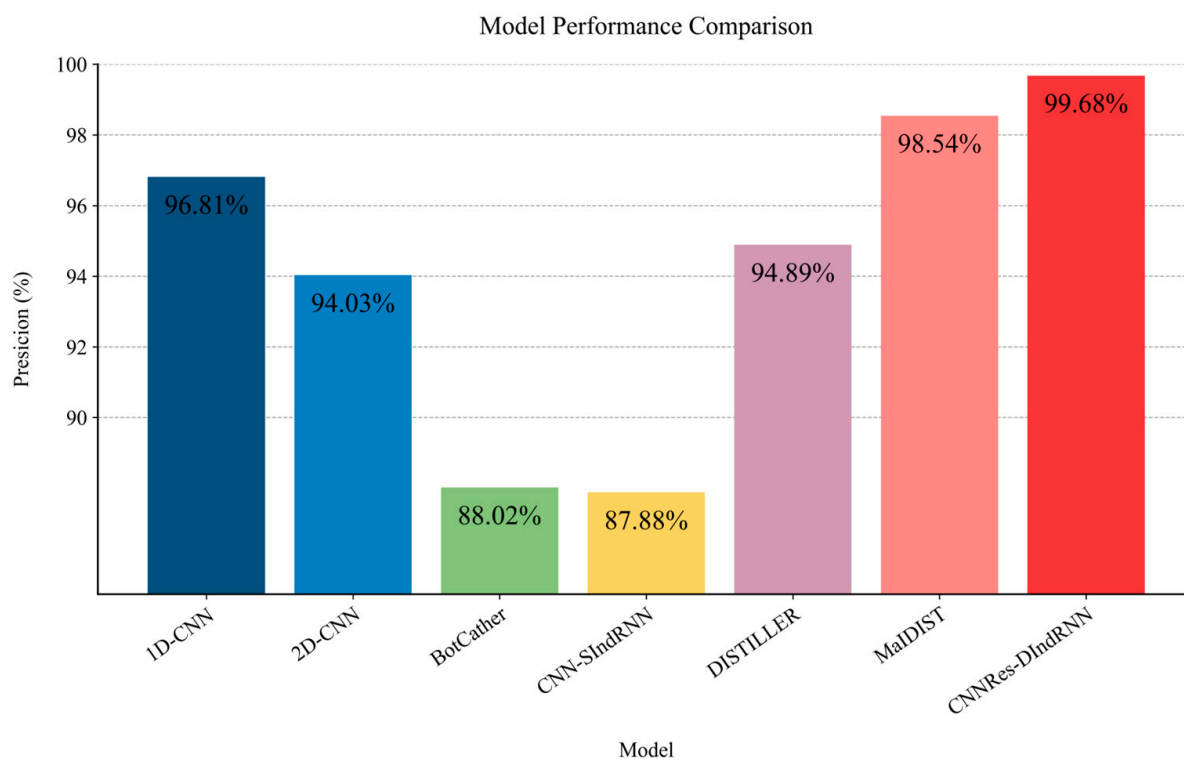


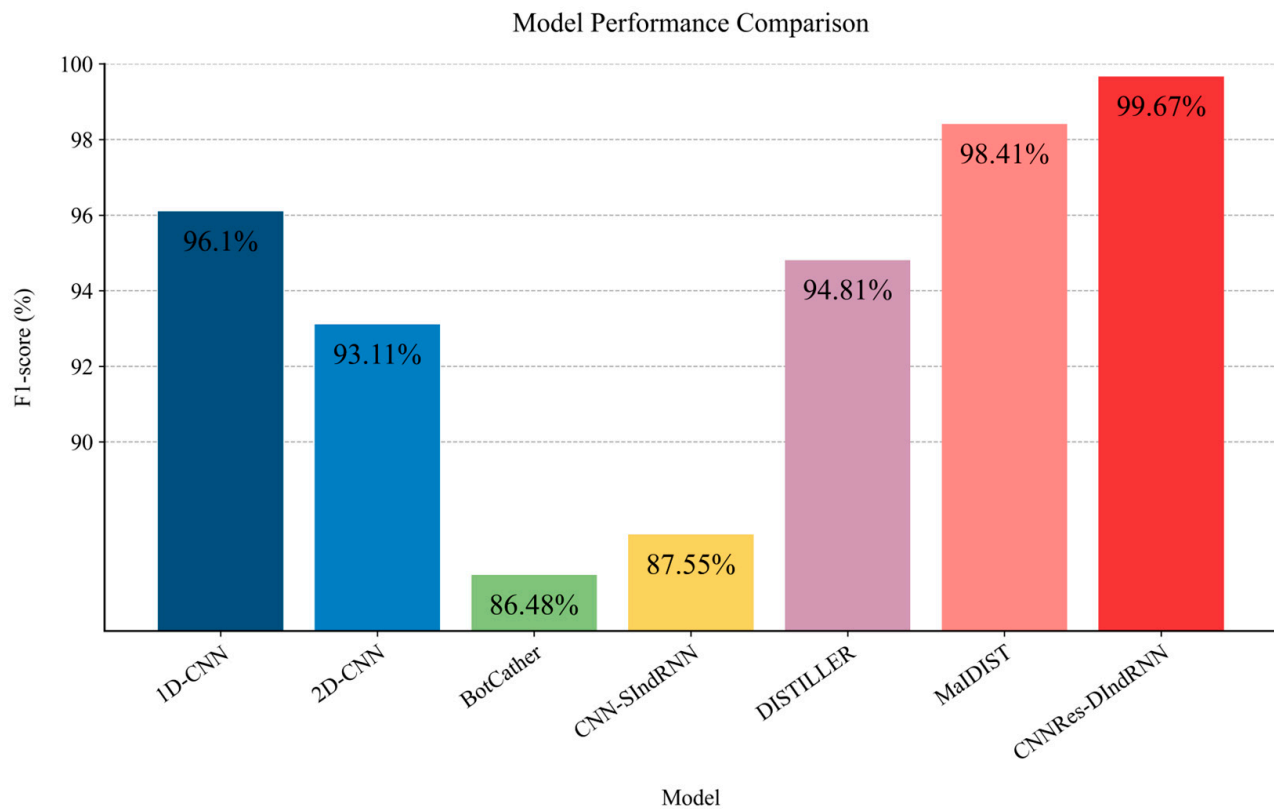
Figure 7. Binary Classification ROC Curve.

4.4.3. Comparison of Multi-Classification Experiment Results

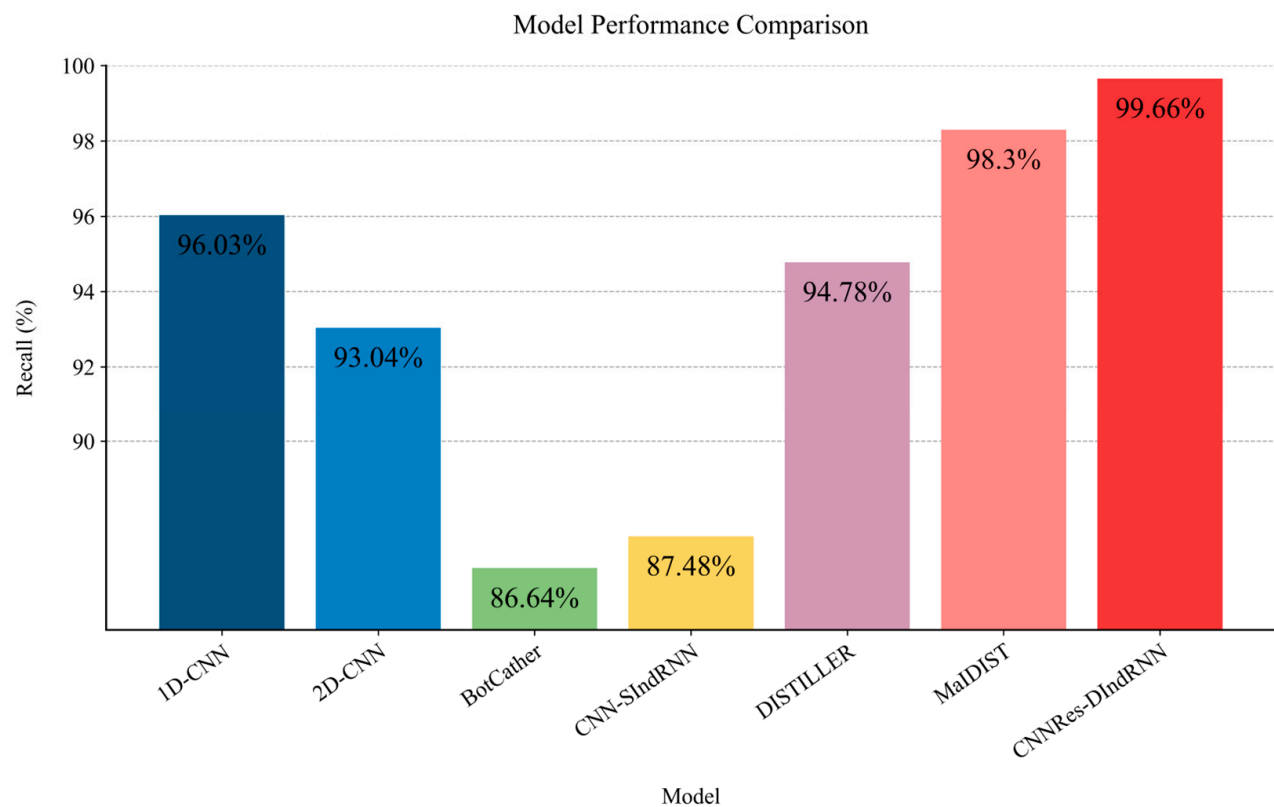
Since real-world network traffic typically contains multiple types of data, binary classification experiments alone may not fully evaluate a model's generalization capability. Therefore, we further conducted multi-classification experiments. The datasets used in this experiment include CIC-IDS2017, MCFP, CTU-13, and MCFP-Normal. All datasets were encoded using XLNet. The specific types of malicious traffic involved are illustrated in Figure 3. The evaluation results of the nine models after training and testing are shown in Figure 8. The experimental results show that the CNN-SIndRNN and BotCather models did not reach 90% on any of the four evaluation metrics. The red part of the four graphs indicates the CNN-Res-DIndRNN model of this experiment, which has the best results with values above 99.60%.

After model comparison, it was found that the CNNRes-DIndRNN model achieved the best performance in classification and detection tasks, with overall results outperforming those of other models. Specifically, the model showed improvements ranging from 0.93% to 12.26% in accuracy, 1.14% to 11.80% in precision, 1.36% to 13.02% in recall, and 1.26% to 13.19% in F1-score compared with other models. The above results fully verify the effectiveness and superiority of the CNNRes-DIndRNN model in this experiment.

**(a) Model Accuracy****(b) Model Precision****Figure 8.** *Cont.*



(c) Model Recall



(d) Model F1-Score

Figure 8. Results of multi-classification experiments.

In the multi-classification comparison experiment, the recognition results of seven baseline models for specific traffic categories are summarized in Table 12. Bold values in the table indicate the highest F1-score within each group. In addition to the F1 values of Bengin and Zeus categories of CNNRes-DIndRNN model are 0.97% and 0.05% lower than those of MaIDISTM model, the F1 values of the other seven traffic types are higher than those of other models in this category. It shows that under the same experimental conditions, the comprehensive recognition ability of CNNRes-DIndRNN model is better than that of other models.

Table 12. F1 Score Comparison Across 9 Traffic Types.

Classification Model	F1-Score								
	Bengin	Artemis	Bunitu	Dridex	Htbot	Neris	Razy	TrickBot	Zeus
1D-CNN [38]	99.48	99.89	98.13	99.58	97.99	99.92	84.09	87.06	98.79
2D-CNN [39]	98.48	99.89	87.74	98.04	85.49	99.03	83.39	87.04	98.79
BotCather [40]	76.57	99.99	39.12	98.71	70.79	96.26	97.67	99.76	99.43
CNN-SIndRNN [43]	70.61	99.15	66.24	91.88	74.06	96.14	96.99	93.15	99.72
DISTILLER [42]	95.84	99.72	81.56	99.22	80.41	98.87	99.08	98.90	99.69
MaIDIST [44]	99.89	99.99	91.34	99.76	95.35	99.62	99.95	99.80	99.99
CNNRes-DIndRNN	98.92	99.99	98.95	99.97	99.25	99.97	99.99	99.99	99.94

Table 13 shows the comparison of parameter count, training time, test time, and model size among five models, with the bolded data indicating the maximum values in each category. Training time is the average duration per iteration on the training samples (1182 samples), and test time is the average duration per test on all test samples. From the table, it can be seen that the CNN-SIndRNN model has the smallest parameter count and model size among all models, with values of 193,737 and 790.0 KB, respectively. However, this model is less efficient in training and testing compared to the CNNRes-DIndRNN model. Overall, although different models have advantages in certain aspects, the CNNRes-DIndRNN model performs best in overall performance.

Table 13. Model Performance Benchmark.

Model	Parameter	Train Time (s)	Test Time (s)	Size
BotCather [40]	2,928,625	1291.37	183.60	22.43 M
CNN-SIndRNN [43]	193,737	116.87	8.35	790.0 k
DISTILLER [42]	3,292,233	129.54	13.70	37.65 M
MaIDIST [44]	3,576,169	150.68	15.61	41.08 M
CNNRes-DIndRNN	926,939	62.37	6.17	10.72 M

4.4.4. Binary Classification Model Testing Results

To validate the model's robustness and prevent overfitting, we conducted external validation tests. In this experiment, we used ten categories of malicious traffic that the model had not previously encountered (details in Table 14) along with over 40,000 unused benign data samples from this experiment for verification. The data sources are provided in Table 1.

We processed the dataset as described, selecting 44,000 benign and 44,000 malicious data samples, with the malicious data randomly sampled from each class in Table 15. The 88,000 processed data samples were tested using a binary classification model, and the results are shown in Table 15. The test achieved an accuracy of 95.37%, precision of 93.52%, recall of 97.50%, and F1 score of 95.47%. Despite a slight decline in performance compared to previous experiments, the model still showed strong results on the external dataset, confirming its robustness and ability to identify unknown malicious data.

Table 14. Additional dataset.

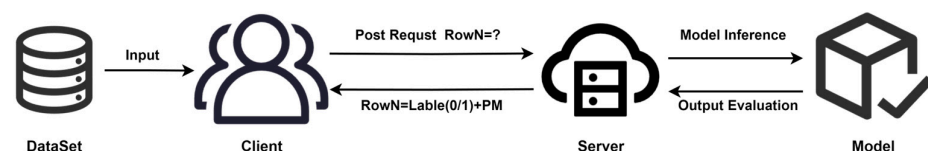
Traffic	Data Sources	Type	Amount
Adload	MCFP	Malware	2
Adware	MCFP	Malware	1975
Conficher	MCFP	Malware	381
Cridex	MCFP	Malware	169,704
Miuref	MCFP	Malware	9246
Opencandy	MCFP	Malware	234
Sality	MCFP	Malware	1753
Shifu	USTC-TF2016	Malware	10
Virut	USTC-TF2016	Malware	738
Wannacry	MCFP	Malware	712

Table 15. Data Detection Results.

Test	Accuracy	Precision	Recall	F1-Score
CNNRes-DIndRNN	95.37%	93.52%	97.50%	95.47%

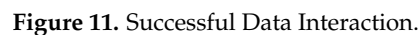
4.4.5. Online Monitoring Experiment Results

Additionally, we conducted an online test to simulate a real-world online monitoring system, using both a local machine and an Ubuntu host. The experiments were conducted in Hebei, China, on a Lenovo Y7000P 2019 with Ubuntu 20.04, an Intel® Core™ i7-9750H CPU @ 2.60 GHz, and an NVIDIA GeForce GTX 1660 Ti GPU. As shown in Figure 9, the diagram represents the overall setup of the online test scenario, with the local machine acting as the data-sending client and the Ubuntu host as the traffic recognition server. We set up a web service using the Flask framework, utilizing Flask's HTTP interface to receive file upload requests from the local machine. The uploaded files are processed on the Ubuntu host, and the results are sent back to the local machine through the HTTP interface. The returned results include traffic labels and performance metrics (throughput and latency)

**Figure 9.** Online Monitoring System.

As shown in Figure 10, we successfully set up the complete online detection system. The local machine's IP address is 192.168.199.1, and the Ubuntu host's IP address is 192.168.199.128. The Flask web application handles the system setup. The figure displays the request logs for sending data files from the local machine, confirming successful data transfer and result retrieval between the local machine and the virtual machine.

During the experiment, we uploaded each row from the CSV file as an independent sample. The file contained 20,000 records—10,000 for malicious traffic and 10,000 for benign traffic—sourced from our ENNetTSd_2025 Dataset and the dataset in Table 14. Each row was packaged into a separate CSV file and uploaded to the Ubuntu host for processing. The results, including processing latency, predicted category, predicted probability, row index, and throughput, were returned to the local machine via an HTTP interface. As shown in Figure 11, this confirms the authenticity of the transmission process.

**Table 16.** Online Validation Evaluation Result 1.

Test	Accuracy	Precision	Recall	F1-Score
CNNRes-DIndRNN	91.95%	92.57%	91.95%	92.26%

Test	Mean-Lat	Min-Lat	Mean-TP	Max-TP
CNNRes-DIndRNN	0.1096 ms	0.0564 ms	9.8808 Req/s	17.74 Req/s

5. Conclusions

This paper proposes an improved method for encrypted malicious traffic classification based on the CNNRes-DIndRNN model. The method uses the Zeek tool (developed by the Zeek Project, USA) to filter TLS-encrypted flows and extracts time-related and packet-level features from PCAP files that adopt TLS encryption. The time-series algorithm NetTiSA is used to calculate temporal features, which are then combined with encryption-related features and encoded by the XLNet encoder to generate a 768-dimensional feature vector. This vector is used as input to the CNNRes-DIndRNN model for experimental analysis.

The experimental results show that in the binary classification task, except for the recall rates of the 1D-CNN and 2D-CNN models being 0.10% and 0.11% higher than that of the CNNRes-DIndRNN model, respectively, the CNNRes-DIndRNN model outperforms all other models in the remaining metrics. In the multi-class classification task, the CNNRes-DIndRNN model achieves the best performance across all four evaluation metrics. However, in the F1-score comparison for individual traffic categories, the MaDIST model performs better in the “Benign” category. Nevertheless, the CNNRes-DIndRNN model achieves higher F1-scores in most of the remaining categories compared to other models.

In the comparison of multi-dimensional model efficiency and resource consumption, the CNNRes-DIndRNN model demonstrated the shortest training and testing times, exhibiting the best overall performance. However, when evaluated individually in terms of parameter count and model size, the model still shows some limitations. Therefore, further improvements are needed to reduce the parameter count and model size of the CNNRes-DIndRNN model.

In the external dataset and online validation experiments, although the four evaluation metrics were lower than in the offline tests, all results exceeded 90%. The throughput and latency metrics in the online scenario validated our approach. They show that the system maintains high accuracy and good real-time responsiveness, proving its feasibility under high load conditions.

Overall, the proposed detection method achieves the shortest training and testing times while maintaining the highest accuracy compared to similar approaches. Among CNN-RNN models, the CNNRes-DIndRNN model excels in parameter count and model size, all while demonstrating optimal robustness. In addition, although this study utilized several public open-source datasets to construct a novel encrypted malicious traffic dataset, the dataset still needs to be expanded in terms of traffic variety. Increasing the diversity of encrypted malicious traffic types will help improve the model’s generalization ability.

Author Contributions: Conceptualization, J.Z. and X.W.; methodology, J.Z.; validation, J.Z., C.L. and G.Y.; formal analysis, X.L. and P.Q.; data curation, F.C., S.L. and R.G.; writing—original draft preparation, C.L. and Q.Z.; writing—review and editing, J.Z. and X.W.; project administration, J.Z. All authors have read and agreed to the published version of the manuscript.

Funding: This work was supported by the Science and Technology Innovation Program for Postgraduate students in IDP subsidized by Fundamental Research Funds for the Central Universities (ZY20250330), the Research on Data Security Encryption Tunnel and Sensitive Data Monitoring and Early Warning System funded by China Metallurgical Geology Bureau (CMGBKY202407), the Research on the feasibility scheme and simulation environment design of unmanned emergency rescue in complex environment in northern Guangdong funded by Shaoguan Data Industry Research Institute, and Exploration and Practice of AI-Empowered Cybersecurity Education Pathways under the Framework of Cultivating Virtue and Talent: Integrating Competition, Teaching, Innovation, Research, and Service.

Data Availability Statement: The data that support the findings of this study are available from the corresponding author upon reasonable request.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Check Point. Cybersecurity Report 2025. 2025. Available online: <https://www.checkpoint.com.cn/support/> (accessed on 27 April 2025).
2. Zscaler ThreatLabz. 2023 ThreatLabz State of Encrypted Attacks Report. 2023. Available online: <https://www.zscaler.com/resources/2023-threatlabz-state-of-encrypted-attacks-report> (accessed on 27 April 2025).
3. Sharafaldin, I.; Lashkari, A.H.; Ghorbani, A.A. Toward generating a new intrusion detection dataset and intrusion traffic characterization. In Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP), Funchal, Portugal, 22–24 January 2018; pp. 108–116. [CrossRef]
4. Stratosphere Laboratory. Stratosphere Laboratory Datasets. 2015. Available online: <https://www.stratosphereips.org/datasets-overview> (accessed on 13 March 2020).
5. García, S.; Grill, M.; Stiborek, J.; Zunino, A. An empirical comparison of botnet detection methods. *Comput. Secur.* **2014**, *45*, 100–123. [CrossRef]
6. Shafiq, M.; Zeadally, S.; Merabti, M. ISCX VPN-NonVPN Traffic Dataset [EB/OL]. Canadian Institute for Cybersecurity. 2018. Available online: <https://www.unb.ca/cic/datasets/vpn.html> (accessed on 10 September 2025).
7. Wang, W.; Zhu, M.; Zeng, X.; Yang, Z. USTC-TFC2016: A Large-Scale Dataset for Network Traffic Classification [Dataset]; University of Science and Technology of China (USTC): Hefei, China, 2017; Available online: <https://github.com/davidyslu/USTC-TFC2016> (accessed on 10 September 2025).
8. The Zeek Project. *Zeek: Open Source Network Security Monitoring*, Version 7.0.0; Zeek Project: Berkeley, CA, USA, 1999. Available online: <https://zeek.org/> (accessed on 10 September 2025).
9. Koumar, J.; Hynek, K.; Pešek, J.; Čejka, T. Nettisa: Extended ip flow with time-series features for universal bandwidth-constrained high-speed network traffic classification. *Comput. Netw.* **2024**, *240*, 110147. [CrossRef]
10. Yang, Z.; Dai, Z.; Yang, Y.; Carbonell, J.; Salakhutdinov, R.; Le, Q.V. XINet: Generalized autoregressive pretraining for language understanding. In Proceedings of the 33rd International Conference on Neural Information Processing Systems, Vancouver, BC, Canada, 8–14 December 2019.
11. Bonfiglio, D.; Mellia, M.; Meo, M.; Rossi, D.; Tofanelli, P. Revealing skype traffic: When randomness plays with you. In Proceedings of the 2007 Conference on Applications, Technologies, Architectures, and Protocols for Computer Communications, Kyoto, Japan, 27–31 August 2007. [CrossRef]
12. Korczyński, M.; Andrzej, D. Markov chain fingerprinting to classify encrypted traffic. In Proceedings of the IEEE INFOCOM 2014-IEEE Conference on Computer Communications, Toronto, ON, Canada, 27 April–2 May 2014. [CrossRef]
13. Husák, M.; Čermák, M.; Jirsík, T.; Čeleda, P. HTTPS traffic analysis and client identification using passive SSL/TLS fingerprinting. *EURASIP J. Inf. Secur.* **2016**, *2016*, 6. [CrossRef]
14. Shen, M.; Wei, M.; Zhu, L.; Wang, M. Classification of encrypted traffic with second-order Markov chains and application attribute bigrams. *IEEE Trans. Inf. Forensics Secur.* **2017**, *12*, 1830–1843. [CrossRef]
15. Anderson, B.; Paul, S.; McGrew, D. Deciphering malware’s use of TLS (without decryption). *J. Comput. Virol. Hacking Tech.* **2018**, *14*, 195–211. [CrossRef]
16. Shekhawat, A.S.; Di Troia, F.; Stamp, M. Feature analysis of encrypted malicious traffic. *Expert Syst. Appl.* **2019**, *125*, 130–141. [CrossRef]
17. de Lucia, M.J.; Cotton, C. Detection of encrypted malicious network traffic using machine learning. In Proceedings of the MILCOM 2019-2019 IEEE Military Communications Conference (MILCOM), Norfolk, VA, USA, 12–14 November 2019. [CrossRef]
18. Dai, R.; Gao, C.; Lang, B.; Yang, L.; Liu, H.; Chen, S. SSL malicious traffic detection based on multi-view features. In Proceedings of the 2019 9th International Conference on Communication and Network Security, Chongqing, China, 15–17 November 2019. [CrossRef]
19. Hu, B.; Zhou, Z.; Yao, L. Malicious traffic detection combining features of packet payload and stream fingerprint. *Comput. Eng.* **2020**, *46*, 163–169. [CrossRef]
20. Ferriyan, A.; Thamrin, A.H.; Takeda, K.; Murai, J. Encrypted malicious traffic detection based on Word2Vec. *Electronics* **2022**, *11*, 679. [CrossRef]
21. Brzuska, C.; Delignat-Lavaud, A.; Egger, C.; Fournet, C.; Kohbrok, K.; Kohlweiss, M. Key-Schedule Security for the TLS 1.3 Standard. In Proceedings of the 28th International Conference on the Theory and Application of Cryptology and Information Security, Taipei, Taiwan, 5–9 December 2022; Springer Nature: Cham, Switzerland, 2022. [CrossRef]
22. Xue, D.; Kallitsis, M.; Houmansadr, A.; Ensafi, R. Fingerprinting Obfuscated Proxy Traffic with Encapsulated {TLS} Handshakes. In Proceedings of the 33rd USENIX Security Symposium (USENIX Security 24), Philadelphia, PA, USA, 14–16 August 2024; Available online: <https://www.usenix.org/conference/usenixsecurity24/presentation/xue-fingerprinting> (accessed on 10 September 2025).

23. Draper-Gil, G.; Lashkari, A.H.; Mamun, M.S.I.; Ghorbani, A.A. Characterization of encrypted and VPN traffic using time-related features. In Proceedings of the 2nd International Conference on Information Systems Security and Privacy (ICISSP), Rome, Italy, 19–21 February 2016. [\[CrossRef\]](#)
24. Vu, L.; Thuy, H.V.; Nguyen, Q.U.; Ngoc, T.N.; Nguyen, D.N.; Hoang, D.T.; Dutkiewicz, E. Time series analysis for encrypted traffic classification: A deep learning approach. In Proceedings of the 2018 18th International Symposium on Communications and Information Technologies (ISCIT), Bangkok, Thailand, 26–29 September 2018. [\[CrossRef\]](#)
25. Shapira, T.; Shavitt, Y. Flowpic: Encrypted internet traffic classification is as easy as image recognition. In Proceedings of the IEEE INFOCOM 2019–IEEE Conference on Computer Communications Workshops (INFOCOM WKSHPS), Paris, France, 29 April–2 May 2019. [\[CrossRef\]](#)
26. Niu, W.; Zhou, J.; Zhao, Y.; Zhang, X.; Peng, Y.; Huang, C. Uncovering APT malware traffic using deep learning combined with time sequence and association analysis. *Comput. Secur.* **2022**, *120*, 102809. [\[CrossRef\]](#)
27. Tang, B.; Wang, C.; Jiang, F.; Zhang, H.; Xu, L.; Zhao, W.; Wang, L.; Li, X. Encrypted traffic classification based on network flow time-space series. *Comput. Appl. Softw.* **2024**, *41*, 297–302. [\[CrossRef\]](#)
28. Koumar, J.; Hynek, K.; Čejka, T. Network traffic classification based on single flow time series analysis. In Proceedings of the 2023 19th International Conference on Network and Service Management (CNSM), Niagara Falls, ON, Canada, 30 October–2 November 2023. [\[CrossRef\]](#)
29. Mikolov, T.; Chen, K.; Corrado, G.; Dean, J. Efficient estimation of word representations in vector space. *arXiv* **2013**, arXiv:1301.3781. [\[CrossRef\]](#)
30. Peters, M.E.; Chen, K.; Corrado, G.; Dean, J. Deep Contextualized Word Representations. In Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, New Orleans, LA, USA, 1–6 June 2018; Association for Computational Linguistics: Kerrville, TX, USA, 2018; Volume 1 (Long Papers), pp. 2227–2237. [\[CrossRef\]](#)
31. Devlin, J.; Chang, M.-W.; Lee, K.; Toutanova, K. Bert: Pre-training of deep bidirectional transformers for language understanding. In Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Minneapolis, MN, USA, 2–7 June 2019; Volume 1 (Long and Short Papers). [\[CrossRef\]](#)
32. Vaswani, A.; Shazeer, N.; Parmar, N.; Uszkoreit, J.; Jones, L.; Gomez, A.N.; Kaiser, Ł.; Polosukhin, I. Attention is all you need. In Proceedings of the 31st International Conference on Neural Information Processing Systems, Long Beach, CA, USA, 4–9 December 2017.
33. Cheng, H.; Xie, J.; Chen, L. CNN-based encrypted C&C communication traffic identification method. *Comput. Eng.* **2019**, *45*, 31–34. [\[CrossRef\]](#)
34. Li, J.; Zhang, H.; Wei, Z. The weighted word2vec paragraph vectors for anomaly detection over HTTP traffic. *IEEE Access* **2020**, *8*, 141787–141798. [\[CrossRef\]](#)
35. Kholgh, D.K.; Kostakos, P. PAC-GPT: A novel approach to generating synthetic network traffic with GPT-3. *IEEE Access* **2023**, *11*, 114936–114951. [\[CrossRef\]](#)
36. Ali, Z.; Tiberti, W.; Marotta, A.; Cassioli, D. Empowering network security: Bert transformer learning approach and mlp for intrusion detection in imbalanced network traffic. *IEEE Access* **2024**, *12*, 137618–137633. [\[CrossRef\]](#)
37. Dong, W.; Yu, J.; Lin, X.; Gou, G.; Xiong, G. Deep learning and pre-training technology for encrypted traffic classification: A comprehensive review. *Neurocomputing* **2024**, *617*, 128444. [\[CrossRef\]](#)
38. Wang, W.; Zhu, M.; Wang, J.; Zeng, X.; Yang, Z. End-to-end encrypted traffic classification with one-dimensional convolution neural networks. In Proceedings of the 2017 IEEE International Conference on Intelligence and Security Informatics (ISI), Beijing, China, 22–24 July 2017. [\[CrossRef\]](#)
39. Wang, W.; Zhu, M.; Zeng, X.; Ye, X.; Sheng, Y. Malware traffic classification using convolutional neural network for representation learning. In Proceedings of the 2017 International Conference on Information Networking (ICOIN), Da Nang, Vietnam, 11–13 January 2017. [\[CrossRef\]](#)
40. Wu, D.; Fang, B.; Cui, X.; Liu, Q. BotCatcher: Botnet detection system based on deep learning. *J. Commun.* **2018**, *39*, 18–28. [\[CrossRef\]](#)
41. Lotfollahi, M.; Jafari Siavoshani, M.; Shirali Hossein Zade, R.; Saberian, M. Deep packet: A novel approach for encrypted traffic classification using deep learning. *Soft Comput.* **2020**, *24*, 1999–2012. [\[CrossRef\]](#)
42. Aceto, G.; Ciunzo, D.; Montieri, A.; Pescapé, A. DISTILLER: Encrypted traffic classification via multimodal multitask deep learning. *J. Netw. Comput. Appl.* **2021**, *183*, 102985. [\[CrossRef\]](#)
43. Li, X.; Xie, X.; Xu, Y.; Zhang, S. Fast Identification Method of Malicious TLS Traffic Based on CNN-SIndRNN. *Comput. Eng.* **2022**, *48*, 148–157+164. [\[CrossRef\]](#)
44. Bader, O.; Lichy, A.; Hajaj, C.; Dubin, R.; Dvir, A. MalDIST: From encrypted traffic classification to malware traffic detection and classification. In Proceedings of the 2022 IEEE 19th Annual Consumer Communications & Networking Conference (CCNC), Las Vegas, NV, USA, 8–11 January 2022. [\[CrossRef\]](#)

45. Huo, Y.; Wu, W.; Zhao, F.; Wang, Q. Multi-view encryption malicious traffic detection method combined with co-training. *J. Xidian Univ.* **2023**, *50*, 139–147. [[CrossRef](#)]
46. Zhao, D.; Yin, Z.; Cui, S.; Lu, Z. Malicious TLS Traffic Detection Based on Graph Representation. *J. Inf. Secur. Res.* **2024**, *10*, 209–215. [[CrossRef](#)]
47. Guo, H.; Chen, Z.; Liu, Z.; Leng, T.; Guo, X.; Zhang, Y. Encrypted traffic classification method based on Low-Dimensional Second-order Markov matrix. *J. Sichuan Univ. (Nat. Sci.)* **2024**, *61*, 36–43. [[CrossRef](#)]
48. Canadian Institute for Cybersecurity. CIC-IDS2018 Dataset [EB/OL]. 2018. Available online: <https://www.unb.ca/cic/datasets/ids-2018.html> (accessed on 10 September 2025).
49. UCI KDD Archive. KDD Cup 1999 Data [Dataset]. 1999. Available online: <https://kdd.ics.uci.edu/databases/kddcup99/kddcup99.html> (accessed on 10 September 2025).
50. Liu, Y.; Ott, M.; Goyal, N.; Du, J.; Joshi, M.; Chen, D.; Levy, O.; Lewis, M.; Zettlemoyer, L.; Stoyanov, V. Roberta: A robustly optimized bert pretraining approach. *arXiv* **2019**, arXiv:1907.11692. [[CrossRef](#)]
51. Lan, Z.; Chen, M.; Goodman, S.; Gimpel, K.; Sharma, P.; Soricut, R. Albert: A lite bert for self-supervised learning of language representations. *arXiv* **2019**, arXiv:1909.11942. [[CrossRef](#)]
52. Majumdar, S. Keras-IndRNN: Implementation of IndRNN in Keras. 2018. Available online: <https://github.com/titu1994/Keras-IndRNN> (accessed on 10 September 2025).
53. Lundberg, S.M.; Lee, S.-I. A unified approach to interpreting model predictions. In Proceedings of the 31st International Conference on Neural Information Processing Systems, Long Beach, CA, USA, 4–9 December 2017.
54. Chen, T.; Carlos, G. XGBoost: A scalable tree boosting system. In Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, San Francisco, CA, USA, 13–17 August 2016. [[CrossRef](#)]
55. Pedregosa, F.; Varoquaux, G.; Gramfort, A.; Michel, V.; Thirion, B.; Grisel, O.; Blondel, M.; Müller, A.; Nothman, J.; Louppe, G.; et al. Scikit-learn: Machine learning in Python. *J. Mach. Learn. Res.* **2011**, *12*, 2825–2830.
56. XGBoost Developers. XGBoost Documentation. 2023. Available online: <https://xgboost.readthedocs.io/> (accessed on 10 September 2025).
57. Hugging Face. Hugging Face Library. Available online: <https://huggingface.co/> (accessed on 10 September 2025).
58. Hugging Face. XLNet Model Documentation. Available online: https://huggingface.co/transformers/model_doc/xlnet.html (accessed on 10 September 2025).
59. Cho, K.; Van Merriënboer, B.; Gulcehre, C.; Bahdanau, D.; Bougares, F.; Schwenk, H.; Bengio, Y. Learning phrase representations using RNN encoder-decoder for statistical machine translation. *arXiv* **2014**, arXiv:1406.1078. [[CrossRef](#)]
60. Hochreiter, S.; Schmidhuber, J. Long short-term memory. *Neural Comput.* **1997**, *9*, 1735–1780. [[CrossRef](#)]
61. Elman, J.L. Finding structure in time. *Cogn. Sci.* **1990**, *14*, 179–211. [[CrossRef](#)]
62. Schuster, M.; Paliwal, K.K. Bidirectional recurrent neural networks. *IEEE Trans. Signal Process.* **1997**, *45*, 2673–2681. [[CrossRef](#)]
63. Graves, A.H.; Schmidhuber, J. Framewise phoneme classification with bidirectional LSTM networks. In Proceedings of the 2005 IEEE International Joint Conference on Neural Networks, Montreal, QC, Canada, 31 July–4 August 2005; Volume 4. [[CrossRef](#)]
64. Li, S.; Li, W.; Cook, C.; Zhu, C.; Gao, Y. Independently recurrent neural network (IndRNN): Building a longer and deeper RNN. In Proceedings of the 2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition, Salt Lake City, UT, USA, 18–23 June 2018. [[CrossRef](#)]

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.